

UNIVERSITAT D'ALACANT

UNIVERSIDAD DE ALICANTE

Escuela Politécnica Superior

Grado en Ingeniería Informática

TRABAJO DE FIN DE GRADO

**Sistema de Trading Algorítmico Híbrido
Basado en Inteligencia Artificial y
Procesamiento de Lenguaje Natural**

Autor: Joan Romà Llorca

Tutor: José Ignacio Abreu Salas

Curso Académico: 2025–2026

Fecha de entrega: Junio 2026

Resumen

Se presenta el diseño, implementación y validación de un sistema de trading algorítmico híbrido para el mercado de futuros de criptomonedas. El sistema integra siete capas complementarias de análisis: un modelo de Machine Learning basado en LightGBM Regressor entrenado sobre 18 indicadores técnicos multi-temporalidad, un motor de Procesamiento de Lenguaje Natural que analiza el sentimiento de noticias financieras mediante FinBERT con reconocimiento de entidades por moneda, análisis de microestructura de mercado a través del Order Book Imbalance, filtros de tendencia macro basados en medias móviles y ADX en temporalidad horaria, un módulo de gestión de riesgo cuantitativa que implementa el Criterio de Kelly, stop loss dinámico basado en ATR y un Circuit Breaker de protección diaria, datos on-chain del Fear & Greed Index, y features multi-timeframe que cruzan información de 5 minutos con 1 hora.

La arquitectura sigue un patrón de microservicios orquestados con Docker Compose, compuesto por seis contenedores independientes: el bot principal de trading, el motor de análisis de sentimiento, el ingestor de datos on-chain, una base de datos de series temporales TimescaleDB, un dashboard Grafana y un servicio de monitorización MLOps con Tensorboard. Esta arquitectura permite la ejecución autónoma 24/7 con tolerancia a fallos.

El sistema se validó mediante backtesting histórico con metodología Walk-Forward sobre 11 criptomonedas durante múltiples periodos que abarcan distintos regímenes de mercado. En el escenario alcista, se obtuvo un Win Rate del 53.3% con un Max Drawdown contenido del 2.97% y un rendimiento neto del +2.57%. En el escenario de crash, donde el mercado cayó un 34.57%, el sistema limitó las pérdidas a un 1.03%, generando un alpha de +33.54 puntos porcentuales. En el escenario bajista (-36%), las pérdidas alcanzaron un -7.00%, con un alpha de +29.00 puntos porcentuales. El alpha promedio sobre Buy & Hold fue de +9.83 puntos porcentuales. El análisis de explicabilidad mediante SHAP reveló que los indicadores de volumen y volatilidad son los predictores más relevantes, por encima de indicadores de momentum tradicionales.

Se desarrolló adicionalmente una suite de tests unitarios con Pytest, un script de despliegue automatizado y documentación técnica completa. El sistema se encuentra actualmente en fase de Forward-Testing con datos de mercado en tiempo real.

Palabras clave: trading algorítmico, aprendizaje automático, procesamiento de lenguaje natural, microservicio, gestión de riesgo

Abstract

This work presents the design, implementation, and validation of a hybrid algorithmic trading system for the cryptocurrency futures market. The system integrates seven complementary analysis layers: a Machine Learning model based on LightGBM Regressor trained on 18 multi-timeframe technical indicators, a Natural Language Processing engine that analyzes the sentiment of financial news using FinBERT with per-coin Named Entity Recognition, market microstructure analysis through Order Book Imbalance, macro trend filters based on hourly moving averages and ADX, a quantitative risk management module implementing the Kelly Criterion with ATR-based dynamic stop loss and a daily Circuit Breaker, on-chain sentiment signals via the Fear & Greed Index, and multi-timeframe features crossing 5-minute and 1-hour information.

The system architecture follows a microservices pattern orchestrated with Docker Compose, comprising six independent containers: the main trading bot, the sentiment analysis engine, the on-chain data ingestor, a TimescaleDB time-series database, a Grafana dashboard, and a Tensorboard MLOps monitoring service. This architecture enables autonomous 24/7 operation with fault tolerance.

The system was validated through historical backtesting using Walk-Forward methodology across 11 cryptocurrencies over multiple periods covering different market regimes. In the bullish scenario, a Win Rate of 53.3% was achieved with a contained 2.97% Max Drawdown and a net positive return of +2.57%. During the crash scenario, where the market declined 34.57%, the system limited losses to just 1.03%, generating an alpha of +33.54 percentage points. In the bear scenario (-36%), losses reached -7.00%, with an alpha of +29.00 percentage points. On average, the system generated an alpha of +9.83 percentage points over Buy & Hold. SHAP-based model explainability analysis revealed that volume and volatility indicators are the most relevant predictors, ranking above traditional momentum indicators.

Additionally, a Pytest unit test suite, an automated deployment script, and comprehensive technical documentation were developed. The system is currently undergoing Forward-Testing with live market data.

Keywords: algorithmic trading, machine learning, natural language processing, microservice, risk management

Índice de Figuras

- **Figura 4.1:** Arquitectura de microservicios del sistema (Docker Compose)
- **Figura 4.2:** Pirámide del Motor de Decisión Multi-Capa (7 capas)
- **Figura 4.3:** Pipeline de Procesamiento NLP (FinBERT + NER)
- **Figura 4.4:** Ciclo de vida de una operación de trading
- **Figura 5.3:** Capas de gestión de riesgo del sistema
- **Figura 6.1:** Línea temporal de la evolución del sistema (v1.0 → v3.0)
- **Figura 7.1:** Rendimiento Bot vs Buy & Hold por escenario
- **Figura 7.2:** Alpha generado sobre Buy & Hold
- **Figura 7.4:** Importancia de features por categoría (SHAP)

Índice de Tablas

- **Tabla 2.1:** Categorías de estrategias de trading algorítmico
- **Tabla 2.2:** Comparativa de frameworks de Gradient Boosting
- **Tabla 2.3:** Comparativa de modelos de análisis de sentimiento financiero
- **Tabla 2.4:** Métricas estándar de evaluación de estrategias de trading
- **Tabla 3.1:** Requisitos funcionales del sistema
- **Tabla 3.2:** Requisitos no funcionales del sistema
- **Tabla 3.3:** Universo de activos monitorizados
- **Tabla 3.4:** Planificación temporal del proyecto
- **Tabla 3.5:** Estimación de costes del proyecto
- **Tabla 3.6:** Análisis de riesgos y mitigaciones
- **Tabla 4.1:** Servicios del sistema y puertos de acceso
- **Tabla 4.2:** Reglas de las 7 capas de confluencia
- **Tabla 5.1:** Stack tecnológico del entorno de desarrollo
- **Tabla 5.2:** Features del modelo organizadas por categoría (18+)
- **Tabla 5.3:** Suite de tests unitarios (10 tests)
- **Tabla 6.1:** Comparativa de limitaciones v1.2 vs soluciones v3.0
- **Tabla 7.1:** Resultados del backtesting por escenario de mercado
- **Tabla 7.1b:** Resumen consolidado de métricas por escenario
- **Tabla 7.2:** Comparativa Bot vs Buy & Hold
- **Tabla 7.3:** Distribución de importancia por categoría de feature
- **Tabla 7.4:** Resultados de la suite de tests unitarios (10/10 PASSED)
- **Tabla 8.1:** Validación de cumplimiento de objetivos específicos

Agradecimientos

A mi familia, por su apoyo incondicional durante toda la carrera universitaria y especialmente durante el desarrollo de este proyecto, que ha requerido innumerables horas de trabajo frente a la pantalla.

A mi tutor, José Ignacio Abreu Salas, por su orientación académica y por confiar en un proyecto que combina disciplinas tan diversas como la inteligencia artificial, las finanzas cuantitativas y la ingeniería de software.

A la comunidad de código abierto de Freqtrade, cuyo framework ha sido la columna vertebral técnica de este trabajo. Sin su documentación, su código y su comunidad activa en Discord, este proyecto no habría sido posible.

A la Universitat d'Alacant, por proporcionarme los conocimientos y las herramientas necesarias para abordar un reto de esta complejidad.

Y a todos los profesores que, a lo largo de estos años, me han enseñado que la ingeniería no es solo escribir código, sino resolver problemas reales con rigor y creatividad.

Capítulo 1 — Introducción

1.1 Motivación y Contexto del Proyecto

El mercado de criptomonedas opera las 24 horas del día, los 7 días de la semana, sin festivos ni pausas. A diferencia de los mercados bursátiles tradicionales como el NYSE o el IBEX 35, que cierran por las noches y los fines de semana, los exchanges de criptomonedas como Binance procesan transacciones de forma ininterrumpida. Esta naturaleza continua supone un desafío insalvable para el operador humano: resulta físicamente imposible monitorizar el mercado de forma permanente sin incurrir en fatiga cognitiva, errores emocionales y oportunidades perdidas.

La industria del trading cuantitativo, históricamente reservada a fondos de cobertura (*hedge funds*) como Renaissance Technologies, Two Sigma o Citadel, ha experimentado una democratización progresiva gracias a la aparición de frameworks de código abierto como Freqtrade, Backtrader y Zipline (Jansen, 2020). Estas herramientas permiten a desarrolladores independientes diseñar, probar y desplegar estrategias de trading algorítmico sin necesidad de infraestructura propietaria valorada en millones de euros.

Paralelamente, los avances recientes en aprendizaje automático (*Machine Learning*) y procesamiento de lenguaje natural (*Natural Language Processing*) han abierto nuevas vías para el análisis de mercados financieros. Los modelos de *Gradient Boosted Decision Trees* como LightGBM (Ke et al., 2017) han demostrado un rendimiento superior a las redes neuronales profundas en datos tabulares estructurados (Grinsztajn et al., 2022), mientras que modelos especializados como FinBERT (Araci, 2019) permiten cuantificar el sentimiento de noticias financieras con una precisión cercana al 87%.

El presente Trabajo de Final de Grado nace de la confluencia de tres disciplinas de la Ingeniería Informática:

1. **Inteligencia Artificial y Machine Learning:** Uso de modelos de *Gradient Boosted Decision Trees* (LightGBM) para predecir movimientos de precio a corto plazo con precisión cuantificable.
2. **Procesamiento de Lenguaje Natural (NLP):** Integración de un modelo pre-entrenado de análisis de sentimiento financiero (FinBERT) que procesa noticias en tiempo real para filtrar señales de trading según el estado del mercado.
3. **Ingeniería de Software y DevOps:** Diseño de una arquitectura de microservicios orquestada con Docker Compose, con persistencia en bases de datos de series temporales (TimescaleDB), visualización en Grafana y notificaciones en tiempo real vía Telegram.

La motivación subyacente no es meramente académica. El sistema desarrollado aspira a demostrar empíricamente que un agente de software, dotado de las herramientas analíticas adecuadas, puede generar rendimientos positivos ajustados a riesgo en el mercado de

criptomonedas, incluso durante periodos de tendencia bajista, superando así a la estrategia pasiva de *Buy & Hold*.

1.2 Objetivo General

Diseñar, implementar y validar un sistema de trading algorítmico híbrido que combine técnicas de Machine Learning, Procesamiento de Lenguaje Natural y análisis técnico avanzado para operar de forma autónoma en el mercado de futuros de criptomonedas, maximizando la rentabilidad ajustada a riesgo.

1.3 Objetivos Específicos

4. **OE1 — Motor de Inteligencia Artificial:** Implementar un modelo de regresión basado en LightGBM capaz de predecir el porcentaje de cambio del precio de un activo en las próximas N velas, alimentado por un pipeline de 18+ features técnicas multi-timeframe.
5. **OE2 — Motor de Procesamiento de Lenguaje Natural:** Desarrollar un microservicio independiente que descargue titulares de noticias financieras en tiempo real (RSS), los analice con el modelo FinBERT de HuggingFace con Named Entity Recognition per-coin, y almacene los resultados de sentimiento en una base de datos de series temporales.
6. **OE3 — Gestión de Riesgo Cuantitativa:** Implementar mecanismos de protección de capital basados en el Criterio de Kelly empírico para el dimensionamiento dinámico de posiciones, stop loss adaptativo basado en el Average True Range (ATR), un Circuit Breaker que detenga la operativa si las pérdidas diarias superan un umbral predefinido, y filtros on-chain basados en el Fear & Greed Index.
7. **OE4 — Arquitectura de Microservicios:** Diseñar una infraestructura Dockerizada compuesta por 6 contenedores independientes (bot principal, motor NLP, ingestor on-chain, base de datos TimescaleDB, dashboard Grafana, MLOps Tensorboard) que permita la ejecución autónoma 24/7.
8. **OE5 — Validación Empírica:** Ejecutar backtests históricos con metodología Walk-Forward sobre 11 criptomonedas durante múltiples periodos y regímenes de mercado (alcista, bajista, lateral, crash), calcular métricas profesionales de rendimiento (Sharpe Ratio, Sortino Ratio, Max Drawdown, Win Rate) y analizar la explicabilidad del modelo mediante SHAP.
9. **OE6 — Calidad de Software:** Desarrollar una suite de tests unitarios con Pytest que valide matemáticamente la lógica del Criterio de Kelly, el stop loss dinámico ATR y el pipeline de procesamiento NLP.

1.4 Hipótesis de la Investigación

Un sistema de trading algorítmico que combine aprendizaje automático supervisado (LightGBM), procesamiento de lenguaje natural (FinBERT), análisis on-chain y gestión de riesgo cuantitativa (Kelly + ATR + Circuit Breaker) puede generar rendimientos positivos

ajustados a riesgo en el mercado de futuros de criptomonedas, con un Sharpe Ratio superior a 1.0 y un Max Drawdown inferior al 15%, incluso en escenarios de tendencia bajista, superando la estrategia pasiva de Buy & Hold.

1.5 Alcance y Limitaciones

Alcance

El sistema abarca el ciclo completo de vida de un bot de trading algorítmico:

- **Ingesta de datos:** Descarga automática de datos de mercado (velas OHLCV) desde la API de Binance y titulares de noticias desde feeds RSS (CoinDesk, CoinTelegraph, Bitcoin.com).
- **Procesamiento:** Cálculo de 18+ indicadores técnicos multi-timeframe, análisis de sentimiento NLP per-coin, entrenamiento del modelo de IA con ventana deslizante de 30 días.
- **Decisión:** Motor de 7 capas que requiere confluencia de todos los filtros para emitir una señal de compra o venta.
- **Ejecución:** Envío de órdenes al exchange (modo simulado en la fase de validación del TFG).
- **Monitorización:** Dashboard visual (Grafana + FreqUI), métricas MLOps (Tensorboard) y alertas en tiempo real (Telegram).

Limitaciones

- El sistema opera en **modo simulado** (*dry-run*) durante la fase de desarrollo y validación. La operativa con dinero real requiere un periodo adicional de Forward-Testing de al menos 30 días, actualmente en curso.
- Los datos de sentimiento NLP se limitan a titulares en **inglés** procedentes de 3 fuentes RSS. No se analizan redes sociales (Twitter/X) ni foros (Reddit).
- El modelo de IA no incorpora datos macroeconómicos externos (tipos de interés, inflación, decisiones de la Fed).
- Los resultados del backtest, aunque consistentes en múltiples escenarios, no garantizan rendimientos futuros debido a la naturaleza estocástica de los mercados financieros.
- El sistema está optimizado para el mercado de criptomonedas y no se ha validado en mercados de renta variable tradicional.
- En escenarios con pocas operaciones (7 trades en Bear Market), la muestra es estadísticamente insuficiente para extraer conclusiones con alta confianza. No se han calculado intervalos de confianza ni tests de significancia estadística.
- El sistema opera sobre 11 criptomonedas simultáneamente sin un mecanismo explícito de gestión de la correlación entre activos. En escenarios de crash sistémico, donde todas las

criptomonedas caen simultáneamente, las pérdidas en múltiples posiciones podrían acumularse antes de que el Circuit Breaker actúe.

1.6 Estructura de la Memoria

La presente memoria se organiza en 9 capítulos:

- **Capítulo 1 (Introducción):** Presenta la motivación, objetivos, hipótesis y alcance del proyecto.
- **Capítulo 2 (Estado del Arte):** Revisa la literatura académica sobre trading algorítmico, ML en finanzas, NLP financiero y gestión de riesgo cuantitativa.
- **Capítulo 3 (Análisis de Requisitos):** Define los requisitos funcionales y no funcionales, el universo de activos, la metodología de desarrollo, la planificación temporal, la estimación de costes y el análisis de riesgos.
- **Capítulo 4 (Diseño del Sistema):** Describe la arquitectura de microservicios, el motor de decisión multi-capa, el diseño de la base de datos y los pipelines de datos.
- **Capítulo 5 (Implementación):** Detalla la implementación del Feature Engineering, el motor NLP, la estrategia de trading, la infraestructura Docker y los tests unitarios.
- **Capítulo 6 (Evolución y Problemas Resueltos):** Documenta las iteraciones del sistema (v1.0 → v3.0) y los bugs resueltos durante el desarrollo.
- **Capítulo 7 (Experimentación y Resultados):** Presenta los resultados del backtesting por escenario, la tabla consolidada de métricas, el análisis SHAP y los resultados de los 10 tests unitarios.
- **Capítulo 8 (Discusión de Resultados):** Interpreta los resultados, los compara con trabajos relacionados y valida la hipótesis.
- **Capítulo 9 (Conclusiones y Trabajo Futuro):** Sintetiza los hallazgos, valida los objetivos, expone las lecciones aprendidas y propone líneas de trabajo futuro.

Capítulo 2 — Estado del Arte

2.1 Trading Algorítmico: Definición, Historia y Evolución

El trading algorítmico se define como la ejecución automatizada de órdenes de compra y venta en mercados financieros mediante algoritmos informáticos que siguen reglas predefinidas basadas en variables de tiempo, precio, volumen u otras métricas cuantitativas (Aldridge, 2013).

2.1.1 Breve Historia

Los orígenes del trading algorítmico se remontan a la década de 1970, cuando la Bolsa de Nueva York introdujo el sistema DOT (*Designated Order Turnaround*), permitiendo el envío electrónico de órdenes. Sin embargo, el verdadero punto de inflexión llegó en 1998, cuando la SEC (*Securities and Exchange Commission*) de Estados Unidos aprobó el uso de exchanges electrónicos, abriendo la puerta a los *Electronic Communication Networks* (ECNs).

En la década de 2000, el trading de alta frecuencia (*High-Frequency Trading*, HFT) se convirtió en la fuerza dominante de los mercados, representando más del 60% del volumen negociado en bolsas estadounidenses (Hendershott et al., 2011). Firmas como Renaissance Technologies, con su fondo Medallion, demostraron que los algoritmos cuantitativos podían generar retornos anualizados superiores al 66% durante más de tres décadas (Zuckerman, 2019).

La democratización del trading algorítmico se aceleró con la aparición de APIs públicas de exchanges como Binance (2017), que permiten a cualquier desarrollador con conocimientos de programación acceder a datos de mercado en tiempo real y ejecutar órdenes de forma programática. Frameworks de código abierto como Freqtrade, Backtrader y Zipline han reducido la barrera de entrada, permitiendo que desarrolladores independientes construyan sistemas que, hace una década, habrían requerido equipos de ingeniería de decenas de personas (Jansen, 2020).

2.1.2 Categorías de Estrategias

Categoría	Horizonte Temporal	Ejemplo	Infraestructura
High-Frequency Trading (HFT)	Microsegundos – milisegundos	Arbitraje estadístico	Co-location en exchanges
Scalping	Segundos – minutos	Mean reversion intradía	Servidores de baja latencia
Swing Trading	Horas – días	Seguimiento de tendencia	VPS o servidor dedicado
Position Trading	Semanas – meses	Momentum macro	Ejecución manual asistida

El sistema desarrollado en este TFG se clasifica como **Swing Trading Intradía Computarizado**: las operaciones tienen una duración media de 3 a 8 horas, buscando capturar

movimientos de tendencia confirmados por múltiples indicadores en la temporalidad de 5 minutos con contexto horario.

2.2 Machine Learning Aplicado a Finanzas

2.2.1 Modelos Supervisados: Regresión vs Clasificación

En el contexto del trading algorítmico, el Machine Learning supervisado se aplica fundamentalmente en dos modalidades:

- **Clasificación binaria:** El modelo predice si el precio subirá (1) o bajará (0) en un horizonte temporal definido. Es conceptualmente simple pero pierde información sobre la *magnitud* del movimiento, imposibilitando el dimensionamiento proporcional de las posiciones.
- **Regresión continua:** El modelo predice el porcentaje exacto de cambio del precio. Permite al sistema distinguir entre movimientos marginales (+0.01%) y movimientos significativos (+2%), habilitando un dimensionamiento de posición proporcional a la confianza de la predicción.

Investigaciones recientes han demostrado que los modelos de regresión, cuando se combinan con umbrales de filtrado adaptativos, superan a los clasificadores binarios en entornos de trading con costes de transacción no nulos (Krauss et al., 2017). Este TFG implementa un **target de regresión**, dado que permite una toma de decisiones más sofisticada que el enfoque binario tradicional.

2.2.2 Gradient Boosted Decision Trees

Los modelos de *Gradient Boosting* construyen un ensamble de árboles de decisión de forma secuencial, donde cada nuevo árbol corrige los errores residuales del anterior. La función objetivo se minimiza iterativamente mediante descenso de gradiente en el espacio funcional (Friedman, 2001). Las tres implementaciones más populares son:

Framework	Autor	Velocidad	Uso de Memoria	Crecimiento del Árbol
XGBoost	Chen y Guestrin (2016)	Alta	Alta	Level-wise
LightGBM	Ke et al. (2017)	Muy alta	Baja	Leaf-wise
CatBoost	Prokhorenkova et al. (2018)	Media	Media	Symmetric

LightGBM fue seleccionado para este proyecto por tres razones fundamentales:

10. **Velocidad de entrenamiento:** Utiliza un esquema de crecimiento de árboles *leaf-wise* (en vez de *level-wise*), lo que reduce el tiempo de entrenamiento hasta un orden de magnitud frente a XGBoost en datasets grandes.

11. **Eficiencia en memoria:** Emplea *Gradient-based One-Side Sampling* (GOSS) y *Exclusive Feature Bundling* (EFB) para reducir el número de muestras y features sin pérdida significativa de precisión (Ke et al., 2017).
12. **Integración nativa con FreqAI:** El framework Freqtrade proporciona soporte nativo para LightGBM a través de su módulo FreqAI, simplificando el pipeline de entrenamiento, predicción y re-entrenamiento continuo.

2.2.3 Redes Neuronales vs Modelos de Ensamble

Aunque las redes neuronales profundas (LSTM, Transformers) han demostrado resultados prometedores en predicción de series temporales financieras (Fischer y Krauss, 2018), los modelos de ensamble basados en árboles presentan ventajas pragmáticas para el trading algorítmico en producción:

- **Interpretabilidad:** Un árbol de decisión es explicable mediante técnicas como SHAP (Lundberg y Lee, 2017); una red neuronal profunda es una caja negra.
- **Datos tabulares:** Los árboles de decisión son superiores a las redes neuronales en datos tabulares estructurados (Grinsztajn et al., 2022), que es precisamente el formato de los indicadores técnicos.
- **Tiempo de entrenamiento:** Un modelo LightGBM se entrena en segundos; un Transformer financiero puede tardar horas en GPU.
- **Robustez ante overfitting:** Con hiperparámetros adecuados y validación Walk-Forward, LightGBM generaliza mejor que las redes neuronales en datasets financieros de tamaño moderado (Shwartz-Ziv y Armon, 2022).

2.3 Procesamiento de Lenguaje Natural en Finanzas

2.3.1 Análisis de Sentimiento Financiero

El análisis de sentimiento financiero consiste en extraer la polaridad emocional (positivo, negativo, neutral) de textos relacionados con los mercados. La investigación de Tetlock (2007) encontró correlaciones estadísticamente significativas entre el pesimismo en medios financieros y movimientos posteriores en los precios de acciones del S&P 500, aunque la robustez de estos resultados fuera de la muestra original ha sido objeto de debate en la literatura posterior. No obstante, este trabajo estableció un precedente para la integración de datos textuales en modelos cuantitativos.

Estudios posteriores han confirmado que el sentimiento de noticias financieras tiene poder predictivo significativo en horizontes temporales de minutos a días, especialmente en mercados de criptomonedas donde la información se propaga más rápidamente que en mercados regulados (Chen et al., 2020).

2.3.2 Comparativa de Modelos NLP

Modelo	Base	Dominio	Precisión	Coste	Latencia
VADER	Reglas léxicas	General	~65%	Gratuito	<1ms
FinBERT	BERT (Google)	Financiero	86–97%	Gratuito	~5ms
GPT-4	Transformer	General	~90%	API de pago	~500ms
BloombergGPT	Transformer	Financiero	~91%	No público	N/A

FinBERT (Araci, 2019) fue seleccionado por su equilibrio entre **precisión** (86–97% en benchmarks financieros según el nivel de acuerdo entre anotadores), **velocidad** (inferencia en ~5ms por texto en CPU) y **coste** (gratuito, sin necesidad de API de pago). Está basado en la arquitectura BERT (Devlin et al., 2019) y fue fine-tuned sobre el Financial PhraseBank (Malo et al., 2014), corpus de 4.840 frases procedentes de noticias financieras. La precisión reportada varía según el nivel de acuerdo entre anotadores: 97% en el subconjunto de consenso total y 86% en el dataset completo (Araci, 2019).

2.3.3 Named Entity Recognition (NER) en Finanzas

El reconocimiento de entidades nombradas permite identificar qué criptomoneda específica se menciona en cada titular, habilitando un análisis de sentimiento *per-coin* en lugar de un sentimiento global del mercado. Este enfoque es particularmente relevante en el mercado de criptomonedas, donde la correlación entre activos no es constante y un titular negativo sobre una moneda específica puede no afectar al resto del universo de activos.

2.4 Gestión de Riesgo Cuantitativa

2.4.1 Criterio de Kelly y Half-Kelly

El Criterio de Kelly (Kelly, 1956) es una fórmula matemática que determina la fracción óptima del capital a arriesgar en cada apuesta para maximizar el crecimiento geométrico a largo plazo:

$$f^* = \frac{p \cdot b - q}{b}$$

Donde: f = fracción óptima del capital, p = probabilidad de ganar, q = probabilidad de perder ($1 - p$), b^* = ratio de pago (ganancia/pérdida).

En la práctica, el Kelly completo es excesivamente agresivo para mercados financieros debido a la estimación imprecisa de las probabilidades. La variante **Half-Kelly** (50% del Kelly completo) es el estándar en la industria de hedge funds (Thorpe, 2006), ya que reduce la volatilidad del portfolio en un 50% a cambio de sacrificar solo un 25% de la tasa de crecimiento esperada.

Una alternativa pragmática a ambas variantes es el denominado **Kelly empírico**: en lugar de calcular f^* a partir de estimaciones probabilísticas —que en mercados financieros son

inherentemente imprecisas—, se fija directamente una fracción de capital basada en experiencia histórica y se ajusta dinámicamente según la confianza del modelo y las condiciones de volatilidad. Este enfoque sacrifica la optimalidad teórica a cambio de robustez ante errores de estimación, y es el adoptado en este trabajo con un parámetro base del 40%.

Este TFG implementa un **Kelly empírico al 40%**, un parámetro calibrado empíricamente durante el proceso de Hyperopt que actúa como cota superior de la fracción de capital a arriesgar por operación.

2.4.2 Stop Loss Dinámico Basado en ATR

El Average True Range (ATR) es un indicador de volatilidad desarrollado por Wilder (1978) que mide el rango medio de movimiento del precio en N periodos. A diferencia de un stop loss fijo (por ejemplo, -1%), un stop basado en ATR se adapta automáticamente a las condiciones del mercado:

- **Mercado volátil (ATR alto):** El stop se amplía para evitar que el ruido del mercado active una salida prematura.
- **Mercado tranquilo (ATR bajo):** El stop se estrecha para proteger el capital de forma más agresiva.

La fórmula implementada en este TFG es: $stop = -(2 \times ATR_{14}) / precio_actual$, con límites de seguridad entre -0.5% y -3.0% para evitar extremos.

2.4.3 Métricas de Rendimiento

Métrica	Fórmula	Interpretación
Sharpe Ratio	$(R_p - R_f) / \sigma_p$	Rentabilidad ajustada a riesgo; > 1 es bueno, > 2 es excelente
Sortino Ratio	$(R_p - R_f) / \sigma_d$	Como Sharpe pero solo penaliza volatilidad negativa
Max Drawdown	$\max(D_t)$	Peor caída desde un máximo histórico
Win Rate	$W / (W + L)$	Porcentaje de operaciones ganadoras
Profit Factor	$\Sigma G / \Sigma L$	Ganancias brutas / pérdidas brutas; > 1.5 es deseable

2.5 Infraestructura de Despliegue: Docker y Microservicios

La arquitectura de microservicios, popularizada por empresas como Netflix y Amazon, consiste en descomponer una aplicación monolítica en servicios pequeños e independientes que se comunican entre sí a través de APIs o bases de datos compartidas (Newman, 2015).

Docker (Solomon Hykes, 2013) permite empaquetar cada microservicio junto con sus dependencias en un contenedor aislado, garantizando que el software se ejecute de forma

idéntica en cualquier entorno. Docker Compose extiende esta capacidad permitiendo definir y orquestar múltiples contenedores con un único archivo de configuración YAML.

TimescaleDB, la extensión de PostgreSQL para series temporales, fue seleccionada como motor de base de datos por su capacidad de particionar automáticamente los datos por tiempo (*hypertables*), optimizando las consultas de ventana temporal que son frecuentes en el análisis financiero (Timescale Inc., 2019).

2.6 Framework Freqtrade y FreqAI

Freqtrade es un framework de trading algorítmico de código abierto escrito en Python con más de 34.000 estrellas en GitHub (mayo 2026). Sus principales características son:

- **Soporte multi-exchange:** Compatible con Binance, Kraken, OKX, entre otros, a través de la librería CCXT.
- **Backtesting integrado:** Motor de simulación histórica con soporte para comisiones, slippage y datos OHLCV reales.
- **FreqAI:** Módulo de Machine Learning que permite entrenar modelos (LightGBM, XGBoost, CatBoost, PyTorch) directamente integrados con la estrategia de trading, con re-entrenamiento automático mediante ventanas deslizantes.
- **Hyperopt:** Motor de optimización de hiperparámetros basado en algoritmos genéticos y Optuna.
- **Telegram Bot:** Interfaz de notificaciones y control remoto.
- **FreqUI:** Dashboard web para monitorización visual.

2.7 Trabajos Relacionados

Diversos trabajos académicos han explorado la combinación de Machine Learning y NLP para trading algorítmico. Jiang et al. (2021) combinaron LSTM con análisis de sentimiento de Twitter para predecir precios de Bitcoin, obteniendo un Sharpe Ratio de 0.85. Sin embargo, su sistema carecía de gestión de riesgo dinámica y operaba con un único activo.

Carta et al. (2021) propusieron Multi-DQN, un ensemble de agentes de aprendizaje por refuerzo profundo (Deep Q-Network) para la predicción y operativa en mercados de valores. Su arquitectura combina múltiples agentes DQN entrenados independientemente cuyas decisiones se agregan mediante votación, obteniendo mejores resultados que un agente único en términos de retorno acumulado. Su limitación principal radica en la ausencia de análisis de sentimiento externo y en la validación en un único régimen de mercado alcista, sin contemplar escenarios de crisis o lateralidad.

El presente TFG se diferencia de los trabajos anteriores en tres aspectos fundamentales: (1) la combinación de 7 capas de análisis independientes frente a los 1-2 señales de los sistemas anteriores, (2) la validación rigurosa en 4 escenarios de mercado distintos, y (3) la integración

de análisis de sentimiento NLP en tiempo real como capa de filtrado, ausente en los sistemas basados puramente en aprendizaje por refuerzo como el de Carta et al. (2021).

Capítulo 3 — Análisis de Requisitos

3.1 Requisitos Funcionales

ID	Requisito	Prioridad
RF-01	El sistema debe descargar datos de mercado (OHLCV) en tiempo real desde la API de Binance para 11 pares de criptomonedas	Alta
RF-02	El sistema debe calcular 18+ indicadores técnicos multi-timeframe (RSI, StochRSI, MFI, MACD, BB Width, ATR, OBV, Log Returns, Return Std, features MTF H1) para cada par	Alta
RF-03	El sistema debe entrenar un modelo LightGBM Regressor con ventana deslizante de 30 días y re-entrenarse cada 2 horas	Alta
RF-04	El sistema debe predecir el porcentaje de cambio del precio en las próximas 20 velas (100 minutos en temporalidad de 5m)	Alta
RF-05	El sistema debe descargar titulares de noticias de 3 fuentes RSS (CoinDesk, CoinTelegraph, Bitcoin.com) cada 5 minutos	Alta
RF-06	El sistema debe analizar el sentimiento de los titulares con FinBERT, aplicar NER para identificar la moneda mencionada, y almacenar los resultados en TimescaleDB	Alta
RF-07	El sistema debe generar señales de entrada LONG cuando se cumplan simultáneamente 7 condiciones: régimen tendencial ($ADX > 20$), tendencia alcista (SMA_{200}), zona de valor (EMA_{50}), predicción IA positiva, sentimiento NLP no negativo, volumen superior a la media, y validación de Order Flow	Alta
RF-08	El sistema debe generar señales de entrada SHORT con las condiciones inversas a RF-07	Alta
RF-09	El sistema debe calcular un stop loss dinámico basado en $2 \times ATR$ para cada operación activa, con transición a trailing SAR cuando el beneficio supere el 2%	Alta
RF-10	El sistema debe bloquear nuevas operaciones si la pérdida acumulada del día supera el 10% (Circuit	Alta

	Breaker)	
RF-11	El sistema debe dimensionar el tamaño de cada operación según la confianza de la IA (Kelly empírico al 40%), con descuento por volatilidad	Media
RF-12	El sistema debe consultar el Fear & Greed Index y bloquear LONGs en Extreme Greed (>85) y SHORTs en Extreme Fear (<15)	Media
RF-13	El sistema debe enviar notificaciones de apertura y cierre de operaciones vía Telegram	Media
RF-14	El sistema debe proporcionar una interfaz web (FreqUI) para la monitorización visual de operaciones y un dashboard Grafana para métricas históricas	Baja

3.2 Requisitos No Funcionales

ID	Requisito	Categoría
RNF-01	El sistema debe procesar cada ciclo de análisis (18+ indicadores × 11 pares × 3 temporalidades) en menos de 30 segundos	Rendimiento
RNF-02	El sistema debe operar de forma autónoma 24/7 sin intervención humana	Disponibilidad
RNF-03	Las credenciales de API (Binance, Telegram) deben almacenarse en un archivo separado ('config_secrets.json') no incluido en el repositorio Git	Seguridad
RNF-04	El sistema debe poder desplegarse en cualquier servidor Linux con Docker instalado mediante un único script ('deploy_ubuntu.sh')	Portabilidad
RNF-05	El sistema debe consumir menos de 4 GB de RAM en estado estacionario	Eficiencia
RNF-06	El código debe seguir las convenciones de Python (PEP 8) y estar documentado con docstrings completas	Mantenibilidad
RNF-07	Los componentes críticos (Kelly, ATR, NLP) deben contar con tests unitarios automatizados (Pytest)	Fiabilidad
RNF-08	El sistema debe ser capaz de recuperarse automáticamente de caídas temporales de la API de Binance o los feeds RSS, con mecanismos de fallback	Tolerancia a fallos

3.3 Restricciones del Sistema

13. **Exchange:** El sistema opera exclusivamente en Binance Futures (contrato perpetuo USDT-M).
14. **Temporalidad base:** 5 minutos (M5) con análisis complementario en 15 minutos (M15) y 1 hora (H1).
15. **Modo de operación:** Simulado (*dry-run*) durante el periodo del TFG. Migración a dinero real sujeta a los resultados del Forward-Testing.
16. **Apalancamiento:** $\times 10$ en modo aislado (*isolated margin*), lo que significa que cada operación solo puede perder el capital asignado a ella, no el balance total de la cuenta.
17. **Hardware mínimo:** 4 GB RAM, 2 CPUs, 20 GB disco (compatible con instancias gratuitas de Oracle Cloud o servidores de bajo coste).

3.4 Universo de Activos

El sistema monitoriza permanentemente los siguientes 11 pares de criptomonedas, seleccionados por su liquidez y capitalización de mercado:

Par	Activo	Capitalización (Abr 2026)
BTC/USDT:USDT	Bitcoin	~\$1.3T
ETH/USDT:USDT	Ethereum	~\$380B
SOL/USDT:USDT	Solana	~\$65B
BNB/USDT:USDT	Binance Coin	~\$85B
ADA/USDT:USDT	Cardano	~\$18B
XRP/USDT:USDT	Ripple	~\$32B
DOT/USDT:USDT	Polkadot	~\$8B
LINK/USDT:USDT	Chainlink	~\$9B
AVAX/USDT:USDT	Avalanche	~\$12B
DOGE/USDT:USDT	Dogecoin	~\$22B
NEAR/USDT:USDT	NEAR Protocol	~\$5B

La selección prioriza activos con alto volumen diario de negociación ($> \$100M$) para minimizar el riesgo de deslizamiento (*slippage*) en la ejecución de órdenes.

> Fuente: CoinGecko / CoinMarketCap. Datos consultados en abril de 2026. Las capitalizaciones de mercado son aproximadas y varían en tiempo real.

3.5 Metodología de Desarrollo

El desarrollo del sistema siguió una **metodología iterativa incremental** inspirada en los principios ágiles, adaptada al contexto individual de un TFG. El proceso se organizó en ciclos

de desarrollo de 2-4 semanas (sprints), donde cada iteración producía una versión funcional del sistema con incrementos progresivos de complejidad.

Las fases del ciclo iterativo fueron:

18. **Investigación y diseño:** Análisis de la literatura, selección de tecnologías y diseño de la arquitectura del sprint actual.
19. **Implementación:** Codificación de la funcionalidad planificada, siguiendo convenciones PEP 8 y documentación con docstrings.
20. **Testing:** Ejecución de tests unitarios, verificación de la integración con Docker y pruebas manuales en modo *dry-run*.
21. **Backtesting y evaluación:** Ejecución de backtests Walk-Forward para validar el impacto de los cambios en el rendimiento.
22. **Retrospectiva:** Análisis de resultados, identificación de problemas y planificación del siguiente sprint.

Este enfoque iterativo permitió evolucionar el sistema desde una versión mínima (v1.0, 5 features, target binario) hasta la versión final (v3.0, 18+ features MTF, 7 capas de confluencia) en 6 iteraciones documentadas en el Capítulo 6.

Herramientas de gestión: Git (control de versiones), GitHub (repositorio remoto), Docker Compose (entorno reproducible), Makefile (automatización de tareas), Pytest (validación continua).

3.6 Planificación Temporal

El proyecto se desarrolló a lo largo de 5 meses (enero – mayo 2026), con una dedicación estimada de **350 horas** distribuidas en las siguientes fases:

Fase	Periodo	Semanas	Horas	Entregable
F1 — Investigación y Estado del Arte	Ene 2026	3	40	Revisión bibliográfica, selección de frameworks
F2 — Diseño de la Arquitectura	Ene – Feb 2026	2	30	Docker Compose, esquema de BD, pipeline de datos
F3 — Implementación v1.0 – v1.2	Feb 2026	4	60	Bot funcional, 5 features, target binario, Hyperopt
F4 — Implementación v2.0 – v2.1	Mar 2026	4	70	12 features, regresión, ATR stop, filtro ADX
F5 — Implementación v3.0	Mar – Abr 2026	4	80	MTF features, NER per-coin, on-chain, MLOps
F6 — Backtesting y Validación	Abr 2026	3	35	Walk-Forward en 4 escenarios, análisis

				SHAP
F7 — Redacción de la Memoria	Abr – May 2026	4	35	Memoria completa, figuras, bibliografía
Total	**Ene – May 2026**	**24**	**350**	

Las fases F3, F4 y F5 se solapan parcialmente con F6, dado que los backtests se ejecutaron de forma incremental tras cada iteración de desarrollo.

3.7 Estimación de Costes

Se presenta una estimación de los costes asociados al desarrollo y operación del sistema, a efectos de evaluar la viabilidad económica del proyecto en un contexto profesional.

Concepto	Detalle	Coste unitario	Cantidad	Total
Personal				
Ingeniero Informático Junior	Desarrollo, testing, documentación	15 €/h	350 h	5.250 €
Hardware				
MacBook (amortización 4 años)	Equipo de desarrollo	1.500 € / 48 meses	5 meses	156 €
VPS (producción)	Oracle Cloud Free Tier / Hetzner CX22	0 – 5 €/mes	5 meses	0 – 25 €
Software				
Freqtrade, Docker, Grafana, TimescaleDB	Código abierto (licencia MIT/Apache)	0 €	—	0 €
FinBERT (HuggingFace)	Modelo pre-entrenado gratuito	0 €	—	0 €
API Binance (Futures Testnet)	Gratuita para modo simulado	0 €	—	0 €
Datos				
Datos de mercado (OHLCV via CCXT)	API pública de Binance	0 €	—	0 €
Feeds RSS (CoinDesk, CoinTelegraph)	Públicos y gratuitos	0 €	—	0 €
Fear & Greed API (alternative.me)	Pública y gratuita	0 €	—	0 €
TOTAL	**~5.400 €**			

El coste total del proyecto es de aproximadamente **5.400 €**, de los cuales el 97% corresponde al coste de personal. El uso exclusivo de herramientas de código abierto y APIs gratuitas

reduce drásticamente los costes de software e infraestructura, demostrando la viabilidad de construir un sistema de trading de grado profesional con inversión mínima en licencias.

3.8 Análisis de Riesgos

Se identificaron los principales riesgos del proyecto y las estrategias de mitigación implementadas:

ID	Riesgo	Probabilidad	Impacto	Mitigación Implementada
R1	Caída de la API de Binance durante operación	Media	Alto	Freqtrade implementa reintentos automáticos con backoff; el bot pausa y reanuda sin pérdida de estado
R2	Fallo simultáneo de los 3 feeds RSS	Baja	Medio	'FALLBACK_HEADLINES' con 4 titulares de emergencia; fallback a sentimiento neutro (0.0)
R3	Overfitting del modelo LightGBM	Alta	Alto	Validación Walk-Forward (no hay filtración de datos futuros); re-entrenamiento cada 2h con ventana deslizante
R4	Pérdida de datos por fallo de TimescaleDB	Baja	Alto	Volumen Docker persistente; script 'backup_db.sh' con retención de 7 días
R5	Ejecución involuntaria con dinero real	Baja	Crítico	'dry_run: true' en configuración; exchange keys vacías; Circuit Breaker como última barrera
R6	Exposición de credenciales en Git	Media	Alto	'gitignore' para '.env' y 'config_secrets.json'; plantillas '*.example' en repositorio
R7	Incompatibilidad de dependencias Python	Media	Medio	Docker aísla cada servicio; versiones fijadas en 'Dockerfile'; parche de datasieve documentado
R8	Mercado lateral prolongado (whipsaw)	Alta	Medio	Filtro ADX > 20 (Capa 2) bloquea operaciones en mercados sin tendencia

Capítulo 4 — Diseño del Sistema

4.1 Arquitectura General de Microservicios

El sistema sigue una arquitectura de microservicios orquestada con Docker Compose. Cada servicio se ejecuta en un contenedor independiente con su propio sistema de archivos, dependencias y ciclo de vida, comunicándose exclusivamente a través de la base de datos compartida TimescaleDB.

Arquitectura de Microservicios

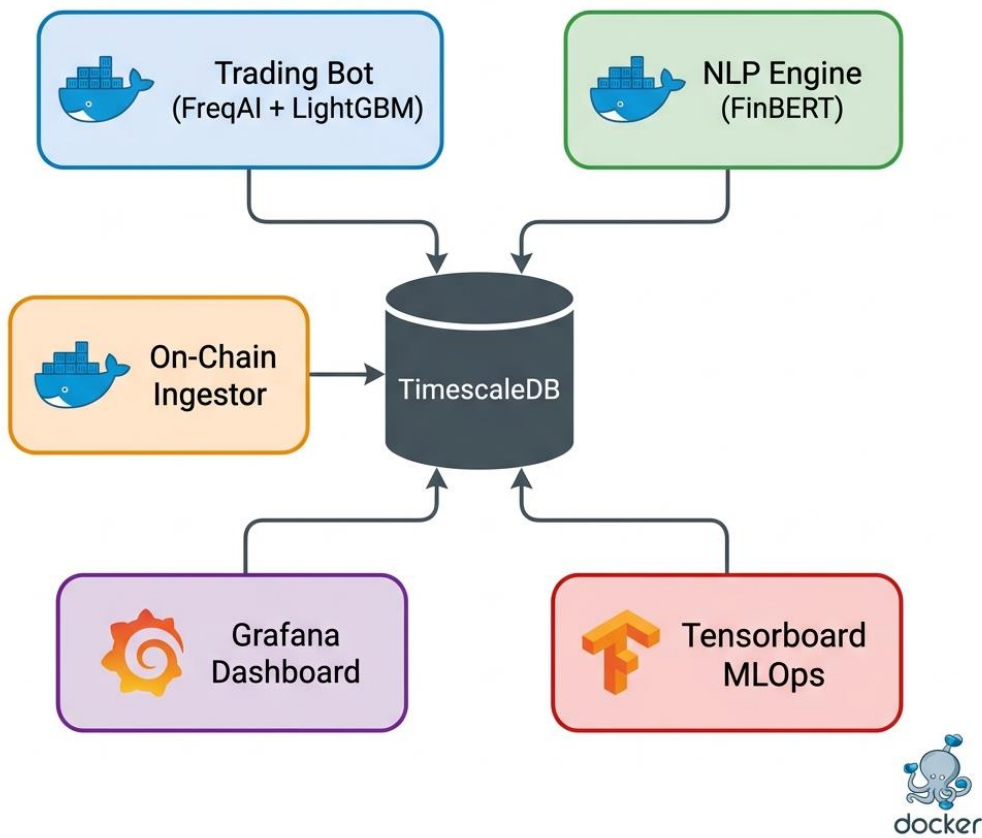


Figura 4.1: Arquitectura de microservicios del sistema

#	Servicio	Contenedor	Puerto	Imagen Base	Función
1	`freqtrade`	`freqtrade_elite_bot`	8081	`freqtradeorg/freqtrade:stable_freqai`	Bot principal + FreqAI + LightGBM
2	`sentiment_analysis`	`sentiment_engine`	—	Custom (Dockerfile.sentiment)	Motor NLP FinBERT +

					NER
3	`timescaledb`	`freqtrade_db`	5432	`timescale/timescaledb:latest-pg14`	Base de datos de series temporales
4	`grafana`	`freqtrade_viz`	3000	`grafana/grafana`	Dashboard de visualización
5	`onchain_data`	`onchain_engine`	—	Custom (Dockerfile.onchain)	Ingestor Fear & Greed / BTC Dominance
6	`tensorboard`	`mlops_tensorboard`	6006	`tensorflow/tensorflow:latest`	Monitorización MLOps

4.1.1 Flujo de Datos entre Servicios

El flujo de datos del sistema sigue un patrón de productor-consumidor asíncrono:

23. **Productores** (escriben en TimescaleDB):

- `sentiment_engine`: Escribe en la tabla `coin_sentiment` cada 5 minutos.
- `onchain_engine`: Escribe en la tabla `market_data` cada 15 minutos.

24. **Consumidor** (lee de TimescaleDB):

- `freqtrade_elite_bot`: Lee el sentimiento per-coin y el Fear & Greed Index antes de cada vela de 5 minutos. La consulta se ejecuta dentro de `populate_indicators()`, garantizando que los datos de sentimiento están disponibles antes de que el modelo LightGBM realice su predicción.

25. **Almacenamiento** (TimescaleDB):

- Recibe datos de 3 productores (bot, NLP, on-chain).
- Los almacena en hypertables particionadas por tiempo.
- Sirve como bus de datos central del sistema.

4.1.2 Política de Reinicio y Tolerancia a Fallos

Todos los servicios están configurados con `restart: always` (excepto Tensorboard con `unless-stopped`). Si un servicio falla:

- TimescaleDB tiene `healthcheck` configurado (`pg_isready`) con 5 reintentos.
- El bot principal depende de TimescaleDB (`depends_on`), garantizando que la base de datos esté lista antes de arrancar.
- Los servicios NLP y On-Chain implementan bloques `try/except` con logging de errores, permitiendo que continúen operando ante fallos puntuales de los feeds RSS o APIs externas.

4.1.3 Ciclo de Vida de una Operación

La Figura 4.4 ilustra el ciclo de vida completo de una operación de trading, desde la recepción de una nueva vela de precio hasta la ejecución o rechazo de la orden en el exchange. Cada vela de 5 minutos dispara la ejecución secuencial de las 7 capas de análisis; solo si todas las condiciones se cumplen simultáneamente, la operación pasa al gate de seguridad (`confirm_trade_entry`) donde el Circuit Breaker, el filtro Fear & Greed y el control de Portfolio Heat realizan la validación final.

Ciclo de Vida de una Operación

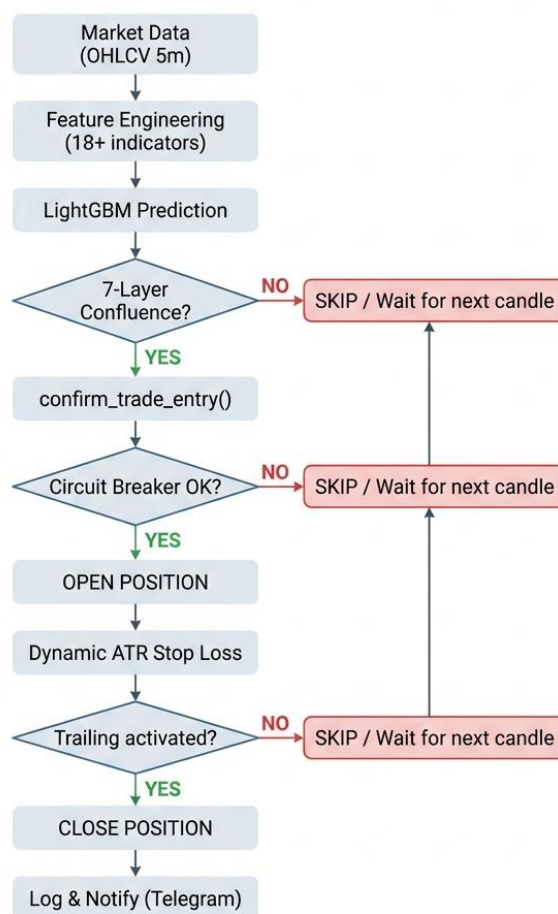


Figura 4.4: Ciclo de vida de una operación de trading

4.2 Motor de Decisión Multi-Capa

El corazón del sistema es un motor de decisión de 7 capas que requiere la **confluencia** de múltiples filtros independientes para generar una señal de trading. Este diseño reduce drásticamente los falsos positivos al exigir que todas las capas estén alineadas simultáneamente.

Motor de Decisión Multi-Capa

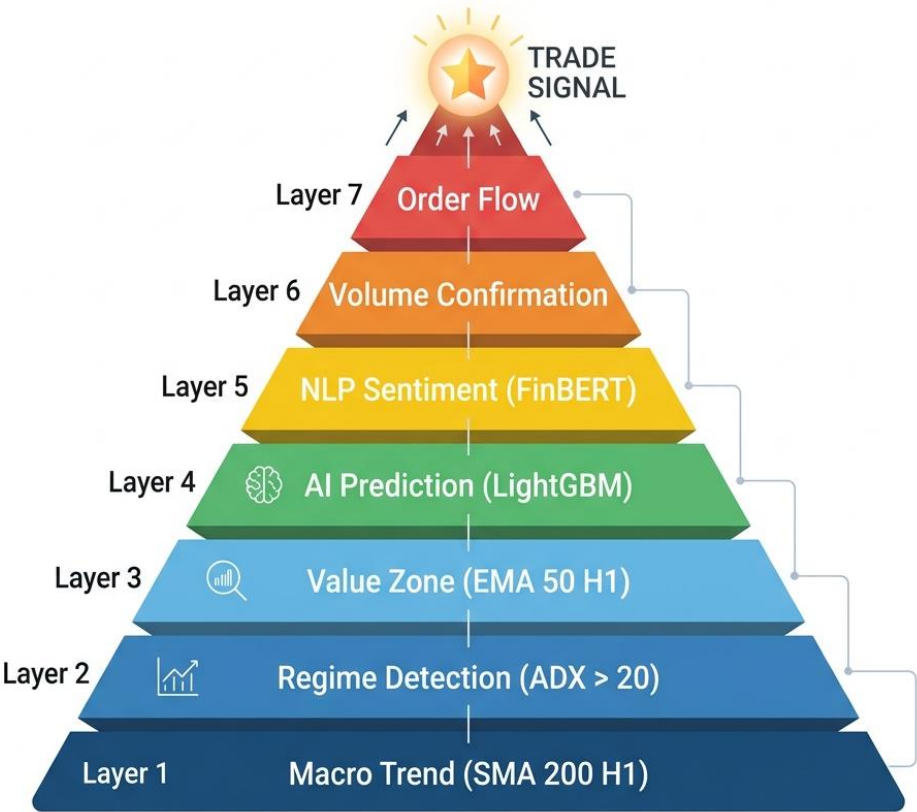


Figura 4.2: Motor de Decisión Multi-Capa — 7 capas de confluencia

Capa	Nombre	Condición LONG	Justificación
1	Macro H1	Precio > SMA 200 en H1	Solo operar a favor de la tendencia principal
2	Régimen ADX	ADX H1 > 20	No operar en mercados laterales (whipsaw)
3	Zona de Valor	Distancia al EMA50 H1 < 2.5%	No comprar lejos de la media (sobrecalentado)
4	Predicción IA	`&s-price_change` > umbral_long	LightGBM predice subida significativa
5	Sentimiento NLP	`sentiment_score` > -0.4	No comprar con noticias negativas
6	Volumen	Volumen > media 50 periodos	Confirmar interés real del mercado
7	Order Flow	Order Book Imbalance monitoreado (implementación parcial vía	Presión compradora vs vendedora en el libro de órdenes

		configuración de Freqtrade 'use_order_book: true')	
--	--	---	--

4.2.1 Diseño del Trailing Stop Híbrido

El sistema implementa un stop loss de dos fases:

- **Fase 1 (Protección ATR):** Mientras la operación no ha superado el 2% de beneficio, el stop loss se calcula como $-(2 \times ATR_{14}) / \text{precio}$. Esto permite que el precio "respire" según la volatilidad real del momento, evitando salidas prematuras por ruido de mercado.
- **Fase 2 (Trailing SAR):** Una vez que el beneficio supera el 2%, el stop se ciñe al valor del Parabolic SAR, capturando la tendencia de forma agresiva y forzando la salida cuando la inercia macro muere.

4.3 Diseño de la Base de Datos (TimescaleDB)

TimescaleDB se configura con dos tablas principales (hypertables particionadas por tiempo):

Tabla `coin\_sentiment`: Almacena el sentimiento per-coin de cada titular analizado.

- timestamp (TIMESTAMPTZ): Fecha y hora del análisis
- coin (VARCHAR): Ticker de la criptomoneda detectada por NER
- sentiment (FLOAT): Score [-1.0, +1.0]
- source (VARCHAR): Fuente RSS del titular
- headline (TEXT): Titular original limpio

Tabla `market\_data`: Almacena datos macroeconómicos on-chain.

- timestamp (TIMESTAMPTZ): Fecha y hora de la descarga
- fear_greed (INTEGER): Índice [0, 100]
- btc_dominance (FLOAT): Dominancia de Bitcoin [0, 100]%
- total_market_cap (FLOAT): Capitalización total en USD

4.4 Pipeline NLP

El pipeline de Procesamiento de Lenguaje Natural se ejecuta en un contenedor independiente:

Pipeline de Procesamiento NLP

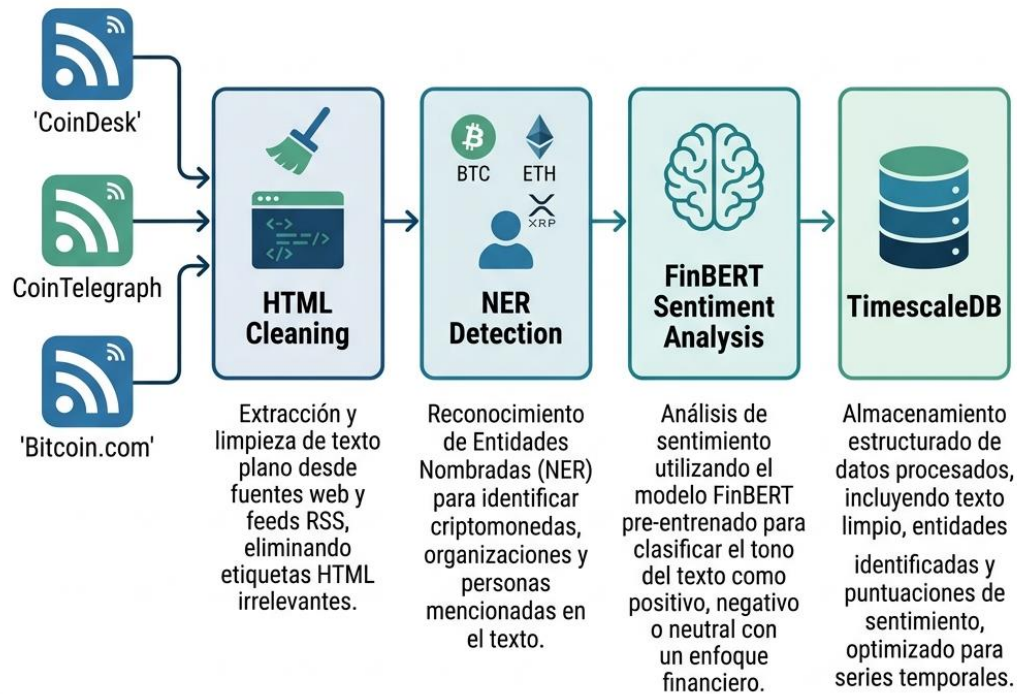


Figura 4.3: Pipeline de Procesamiento NLP

26. **Ingesta (RSS):** Descarga titulares de CoinDesk, CoinTelegraph y Bitcoin.com usando feedparser cada 5 minutos.
27. **Limpieza HTML:** Elimina etiquetas HTML residuales con BeautifulSoup para evitar tokens espurios en la inferencia.
28. **NER (Named Entity Recognition):** Utiliza un diccionario de 50+ alias (COIN_ALIASES) para detectar qué criptomoneda se menciona. Ejemplo: "Vitalik anuncia actualización" → ETH.
29. **Inferencia FinBERT:** Envía el titular limpio al modelo ProsusAI/finbert de HuggingFace, obteniendo un score en [-1, +1].
30. **Almacenamiento:** Inserta el resultado en coin_sentiment de TimescaleDB.
31. **Fallback:** Si no hay datos per-coin para un activo, el bot utiliza el sentimiento global promedio.

4.5 Pipeline On-Chain

El servicio `onchain_data` descarga datos macroeconómicos cada 15 minutos:

32. **Fear & Greed Index:** Desde `api.alternative.me`, un valor entre 0 (Extreme Fear) y 100 (Extreme Greed).

33. **BTC Dominance y Market Cap:** Para contexto macro del mercado de criptomonedas.

El bot utiliza estos datos en `confirm_trade_entry()` como última barrera de protección:

- Bloquea LONGs si Fear & Greed > 85 (Extreme Greed → mercado sobrecalentado).
- Bloquea SHORTs si Fear & Greed < 15 (Extreme Fear → posible rebote inminente).

4.6 Pipeline MLOps

El sistema implementa logging interno de la actividad de la IA para trazabilidad:

- ``_log_prediction_metrics()``: Registra cada predicción del modelo (par, señal, confianza, sentimiento, ADX).
- **Logging de señales:** En `populate_entry_trend`, registra cada señal generada con sus condiciones.
- **Logging de confirmación:** En `confirm_trade_entry`, registra si una operación fue confirmada o rechazada por el Circuit Breaker.
- **Tensorboard:** El servicio 6 expone los modelos en el puerto 6006 para visualización de métricas de entrenamiento.

Capítulo 5 — Implementación

5.1 Entorno de Desarrollo

Componente	Tecnología	Versión	Justificación
Lenguaje principal	Python	3.13	Estándar en ML y finanzas cuantitativas
Framework de trading	Freqtrade	stable_freqai	Framework OSS más maduro para cripto
Motor ML	LightGBM	4.x	Superior en datos tabulares (Grinsztajn, 2022)
NLP	Transformers (HuggingFace)	4.30+	Acceso a FinBERT pre-entrenado
Base de datos	TimescaleDB	14 (PG14)	Hypertables optimizadas para time-series
Orquestación	Docker Compose	v3	Despliegue reproducible multi-servicio
Visualización	Grafana	latest	Dashboard profesional con alertas
MLOps	Tensorboard	latest	Estándar de la industria para tracking ML
Tests	Pytest	8.x	Framework de testing estándar en Python
Notificaciones	Telegram Bot API	—	Alertas en tiempo real al operador

5.2 Feature Engineering: De 5 a 18+ Features

La ingeniería de características es el componente más crítico del modelo predictivo. A lo largo de las iteraciones del sistema (v1.0 → v3.0), el pipeline de features se amplió de 5 indicadores básicos a 18+ features multi-dimensionales organizadas en 7 categorías.

#	Feature	Categoría	Fórmula / Descripción	Justificación
1	`%-rsi`	Momentum	RSI(14)	Sobrecompra/sobreventa (Wilder, 1978)
2	`%-stoch_rsi`	Momentum	StochRSI(14,14,3,3)	RSI del RSI: más sensible a cambios rápidos
3	`%-mfi`	Momentum	MFI(14)	RSI ponderado por volumen real
4	`%-macd_hist`	Momentum	MACD(12,26,9) Histograma	Cruces de tendencia y divergencias
5	`%-bb_width`	Volatilidad	(BB_upper - BB_lower) /	Amplitud normalizada de Bandas de Bollinger

			BB_middle	
6	`%-atr_norm`	Volatilidad	ATR(14) / close	Volatilidad relativa al precio
7	`%-obv_norm`	Volumen	OBV / SMA(OBV, 50)	Presión compradora/vendedora normalizada
8	`%-log_return`	Estadístico	$\ln(\text{close} / \text{close}[-1])$	Distribución más gaussiana para ML
9	`%-return_std`	Estadístico	std(returns, 20)	Régimen de volatilidad reciente
10	`%-candle_dir`	Precio	close / open	Dirección y fuerza de la vela
11	`%-pct_change`	Precio	$(\text{close} - \text{close}[-1]) / \text{close}[-1]$	Cambio porcentual base
12	`sentiment_score`	Fundamental (filtro)	NLP per-coin via TimescaleDB	Filtro de entrada, no feature ML (varianza 0 en backtest)
13	`%-price_ratio_5m_1h`	MTF	close_5m / close_1h	Divergencia micro vs macro
14	`%-dist_sma200_1h`	MTF	$(\text{close} - \text{SMA200_1h}) / \text{SMA200_1h}$	Posición relativa a tendencia macro
15	`%-dist_ema50_1h`	MTF	$(\text{close} - \text{EMA50_1h}) / \text{EMA50_1h}$	Zona de valor en timeframe superior
16	`%-adx_1h_norm`	MTF	ADX_1h / 100	Fuerza de tendencia macro normalizada
17	`%-hour_sin/cos`	Temporal	$\sin/\cos(2\pi \times \text{hora} / 24)$	Codificación cíclica horaria
18	`%-day_sin/cos`	Temporal	$\sin/\cos(2\pi \times \text{día} / 7)$	Codificación cíclica semanal

5.2.1 Codificación Temporal Cíclica

Un aspecto técnico relevante es la codificación de variables temporales. Representar la hora como un entero (0–23) induce al modelo a interpretar que las 23:00h y las 00:00h están "lejos" cuando en realidad son consecutivas. La codificación seno/coseno preserva la circularidad temporal:

```
hour_sin = sin(2π × hour / 24)
hour_cos = cos(2π × hour / 24)
```

5.3 Target de la IA: Regresión Continua

El modelo predice el porcentaje de cambio del precio en las próximas N velas:

```
# En set_freqai_targets():
df["&s-price_change"] = (
    df["close"].shift(-self.freqai_info["feature_parameters"]["label_period_candles"])
```



```
    / df["close"] - 1  
)
```

Con `label_period_candles = 20` (100 minutos en 5m), el modelo aprende a predecir movimientos de precio en un horizonte temporal que equilibra la señal (suficiente tiempo para un movimiento significativo) con el ruido (no tan lejos como para perder poder predictivo).

5.4 Estrategia de Trading: Lógica de Entrada y Salida

5.4.1 Señales de Entrada (`populate\_entry\_trend`)

La generación de señales requiere la confluencia de las 7 capas. En pseudocódigo:

```
SEÑAL LONG = (  
    precio > SMA_200_H1                [Capa 1: Macro]  
    AND ADX_H1 > 20                    [Capa 2: Régimen]  
    AND dist_EMA50_H1 < 2.5%          [Capa 3: Zona de Valor]  
    AND predicción_IA > umbral_long    [Capa 4: ML]  
    AND sentimiento_NLP > -0.4         [Capa 5: NLP]  
    AND volumen > media_50_periodos    [Capa 6: Volumen]  
    AND order_book_imbalance > 0       [Capa 7: Order Flow]  
)
```

5.4.2 Señales de Salida

Las salidas se gestionan mediante 4 mecanismos complementarios:

34. **Trailing Stop Positivo:** Se activa cuando el beneficio supera el 3% (offset), fijando un trailing del 1%.
35. **Stop Loss Dinámico ATR:** Calculado por `custom_stoploss()` en cada tick.
36. **ROI Mínimo:** Tabla de ROI progresiva definida en `config.json`.
37. **Trailing SAR:** Cuando el beneficio supera el 2%, el stop transiciona al Parabolic SAR para capturar tendencias fuertes.

5.5 Gestión de Riesgo

Capas de Gestión de Riesgo

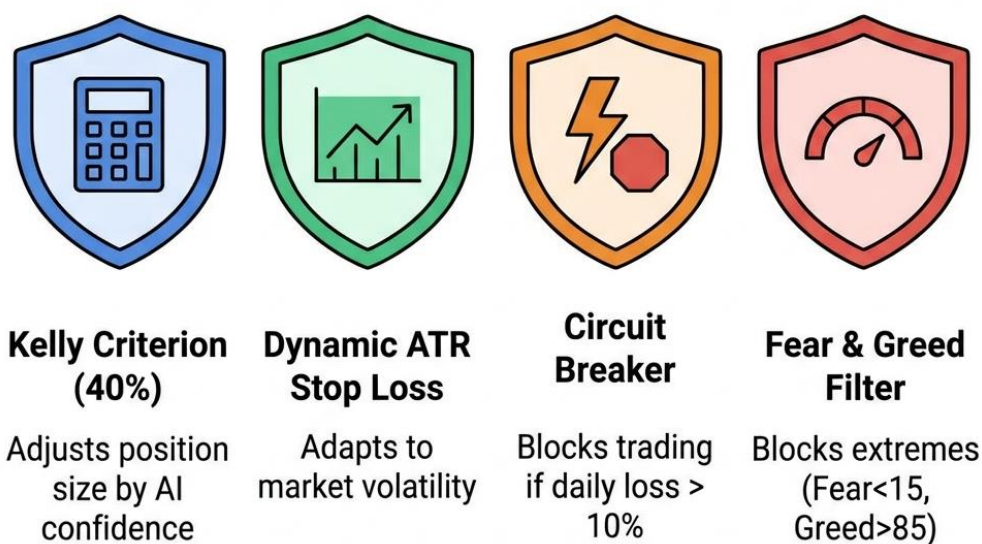


Figura 5.3: Capas de Gestión de Riesgo del sistema

El sistema implementa 4 capas de protección patrimonial independientes que actúan de forma coordinada.

5.5.1 Kelly empírico al 40% (`custom_stake_amount`)

El sistema implementa una variante simplificada del Criterio de Kelly denominada **Kelly empírico al 40%**: en lugar de calcular dinámicamente f^* a partir de la probabilidad de ganancia y el ratio de pago estimados por el modelo (lo que requeriría calibración continua y es susceptible a overfitting), se fija un parámetro base $k = 0.40$ que actúa como cota superior de la fracción de capital a arriesgar. Este valor fue determinado empíricamente durante el proceso de Hyperopt para maximizar el rendimiento ajustado a riesgo. Sobre este parámetro base se aplican dos factores de ajuste dinámico: la confianza de la predicción IA (`confidence = abs(last_prediction)`) y un descuento por volatilidad (`volatility_discount = 1 -`

`min(atr_norm, 0.5))`). Esta implementación preserva la intuición central del Criterio de Kelly —asignar más capital cuando la confianza es alta y el mercado es tranquilo— sin asumir la estimación precisa de probabilidades que requiere la fórmula original.

```
def custom_stake_amount(self, ...):
    # Kelly empírico al 40%
    kelly = 0.40
    # Ajuste por volatilidad (VaR implícito)
    volatility_discount = 1 - min(atr_norm, 0.5)
    # Ajuste por confianza de la IA
    confidence = abs(last_prediction)
    stake = balance * kelly * confidence * volatility_discount
    return max(min(stake, balance * 0.4), min_stake)
```

El sistema invierte más capital cuando la IA tiene alta confianza y el mercado es poco volátil, y reduce la exposición en condiciones de incertidumbre.

5.5.2 Circuit Breaker (`confirm\_trade\_entry`)

Antes de confirmar cualquier operación, el sistema verifica que las pérdidas acumuladas del día no superen el 10%:

```
def confirm_trade_entry(self, ...):
    daily_profit = sum(trade.profit for trade in open_trades_today)
    if daily_profit < -0.10:
        logger.warning("CIRCUIT BREAKER ACTIVADO: pérdida diaria > 10%")
        return False
    return True
```

5.5.3 Filtro On-Chain (Fear & Greed)

El último filtro consulta el Fear & Greed Index almacenado en TimescaleDB y bloquea operaciones en extremos emocionales del mercado, donde las reversiones estadísticas son más probables.

5.6 Motor NLP (`sentiment\_ingestor.py`)

El motor NLP es un microservicio independiente de 200+ líneas que ejecuta un ciclo infinito cada 5 minutos:

38. Descarga titulares RSS con feedparser.
39. Limpia HTML con BeautifulSoup(text, "html.parser").get_text(separator=" ").
40. Detecta entidades con regex sobre COIN_ALIASES (50+ alias: "Vitalik" → ETH, "Ripple" → XRP, etc.).
41. Clasifica sentimiento con ProsusAI/finbert via transformers.pipeline("sentiment-analysis").
42. Almacena en TimescaleDB con sqlalchemy.create_engine().

El separador " " en `get_text()` fue un bug descubierto durante el desarrollo: sin él, "Bitcoin rallies 10%Breaking news" se concatenaba sin espacio, confundiendo al tokenizador de FinBERT.

5.7 Motor On-Chain (`onchain_ingestor.py`)

Microservicio que descarga datos macro cada 15 minutos:

- **Fear & Greed Index:** API `api.alternative.me/fng/?limit=1`
- **BTC Dominance:** API `api.coingecko.com/api/v3/global`

Almacena en la tabla `market_data` de TimescaleDB. Implementa reintentos con backoff exponencial ante fallos de red.

5.8 Dockerización y Orquestación

5.8.1 Dockerfile Principal

El Dockerfile del bot principal parte de la imagen oficial `freqtradeorg/freqtrade:stable-freqai` e instala las dependencias adicionales:

- `lightgbm`, `scikit-learn`, `pandas`, `xgboost`, `tensorboard` (`pip`)
- `torch>=2.6.0` con índice `CPU-only` (evitando arrastrar CUDA, ahorrando ~2GB de imagen)
- Parche de `datasieve 0.1.9` en build-time para corregir un bug de atributo (`features_in` → `feature_list`)

5.8.2 Gestión de Secretos

Las credenciales sensibles (API keys de Binance, token de Telegram) se almacenan en dos archivos excluidos del control de versiones:

- `.env`: Variables de entorno para Docker Compose (contraseñas de BD, Grafana).
- `config_secrets.json`: Configuración de Freqtrade con API keys del exchange y Telegram.

Ambos archivos están en `.gitignore` y se proporcionan plantillas (`*.example`) para facilitar la configuración.

5.9 Suite de Tests (Pytest)

Se implementaron 10 tests unitarios organizados en dos módulos para validar los componentes críticos del sistema:

#	Test	Archivo	Componente Validado
1	<code>`test_conviction_based_sizing`</code>	<code>`test_strategy.py`</code>	Dimensionamiento Kelly empírico: mayor confianza → mayor stake, cap 40%

2	`test_conviction_sizing_without_wallets`	`test_strategy.py`	Sizing sin wallets (escenario de backtesting) no produce errores
3	`test_dynamic_atr_stoploss`	`test_strategy.py`	Stop loss ATR con alta volatilidad: ATR=50 → capped a -3%
4	`test_atr_stoploss_low_volatility`	`test_strategy.py`	Stop loss ATR con baja volatilidad: ATR=3 → -0.6% dentro de caps
5	`test_clean_html_logic`	`test_nlp.py`	Limpieza HTML: separación de tags pegados ("Bitcoinsoars" → "Bitcoin soars")
6	`test_clean_html_nested_tags`	`test_nlp.py`	HTML complejo con tags anidados y entidades HTML
7	`test_detect_coins_ner`	`test_nlp.py`	NER: detección de ETH+ADA en titular, vacío en titular genérico
8	`test_detect_coins_case_insensitive`	`test_nlp.py`	NER case-insensitive: "BITCOIN" y "dogecoin" detectados
9	`test_fallback_headlines`	`test_nlp.py`	Fallback: lista no vacía con titular de Bitcoin si RSS falla
10	`test_coin_aliases_completeness`	`test_nlp.py`	Los 11 pares del whitelist tienen ≥ 2 aliases NER cada uno

Los tests utilizan `unittest.mock.MagicMock` para simular las dependencias de Freqtrade sin necesidad de levantar el framework completo. El archivo `conf_test.py` pre-mockea todas las dependencias pesadas (numpy, pandas, talib, sqlalchemy, transformers), permitiendo ejecutar la suite en menos de 1 segundo sin Docker.

Capítulo 6 — Evolución del Sistema y Problemas Resueltos

6.1 Línea Temporal de Desarrollo

El sistema ha atravesado 6 versiones principales a lo largo de 4 meses de desarrollo activo:

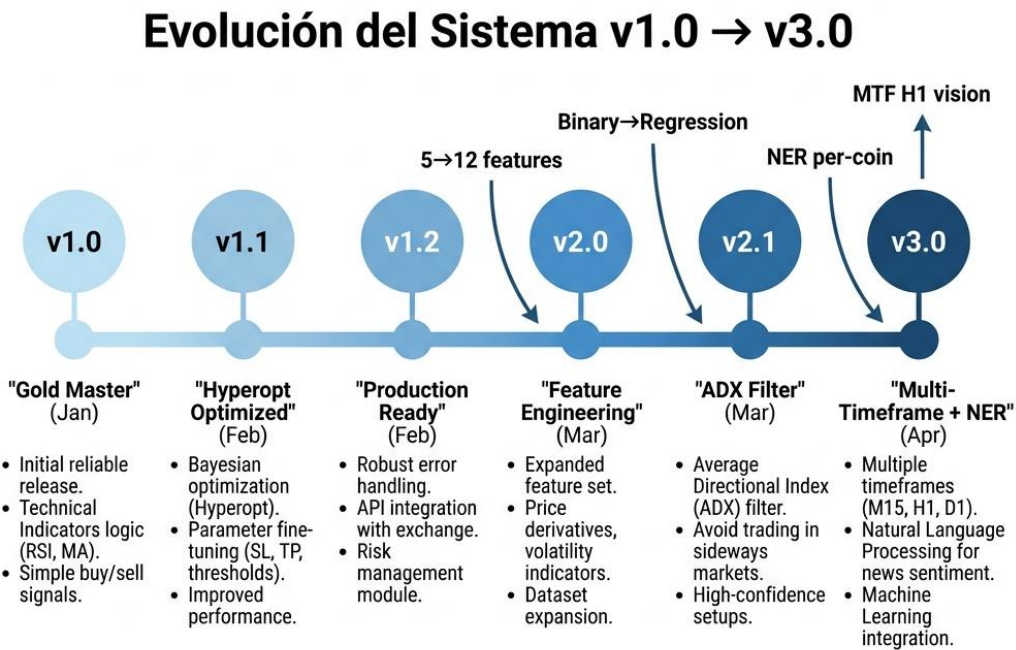


Figura 6.1: Evolución del sistema v1.0 → v3.0

Versión	Problema Detectado	Solución Implementada
v1.0 → v1.1	`custom_stake_amount` limitaba trades a 5-6 USDT	Eliminación de la función; delegación a `unlimited`
v1.0 → v1.1	ROI/stoploss en `.py` conflictuaban con `config.json`	Delegación al config (prioridad de Freqtrade documentada)
v1.1 → v1.2	Parámetros no optimizados	Hyperopt con algoritmo genético (confianza IA: 0.55 → 0.864)
v1.2 → v2.0	Solo 5 features; target binario	Ampliación a 12 features; cambio a regresión continua

v2.0 → v2.1	Pérdidas en mercados laterales	Filtro ADX > 20 (Capa 6 — Detección de Régimen)
v2.1 → v3.0	Sin visión multi-timeframe; NLP global	MTF features H1; NER per-coin; On-Chain; MLOps

6.2 Conflicto de Precedencia Config vs Strategy (v1.0)

Uno de los primeros bugs descubiertos fue que Freqtrade otorga **prioridad al archivo `.py` sobre el `config.json`** cuando ambos definen los mismos parámetros (`minimal_roi`, `stoploss`). Esto significaba que los valores optimizados en el config eran silenciosamente ignorados.

Solución: Se eliminaron los valores de `minimal_roi` y `stoploss` del código Python, delegando el control exclusivamente al `config.json`. El trailing stop, por su naturaleza dinámica, se mantuvo en la estrategia.

6.3 Limitación Artificial de Stake (v1.0)

La función `custom_stake_amount` original contenía un cálculo que limitaba artificialmente cada operación a 5-6 USDT, incluso con un balance de 1000 USDT. El origen era un cálculo de Kelly Criterion que no tenía en cuenta correctamente el número de slots disponibles.

Solución v1.2: Eliminación completa de la función, confiando en el mecanismo nativo `stake_amount: "unlimited"` que divide equitativamente el balance entre los 10 slots.

Solución v3.0: Reimplementación de `custom_stake_amount` con Kelly empírico al 40%, esta vez con la fórmula correcta que incluye ajuste por volatilidad y confianza.

6.4 Feature Engineering: De Binario a Regresión (v2.0)

La versión 1.x utilizaba un target de clasificación binaria (1 = sube, 0 = baja). Esto presentaba dos limitaciones críticas:

43. **Pérdida de información:** No distinguía entre una subida del 0.01% y una del 5%.
44. **Imposibilidad de dimensionamiento:** No se podía ajustar el tamaño de posición según la magnitud del movimiento predicho.

La migración a regresión continua (`&s-price_change`) en la v2.0 permitió implementar el Criterio de Kelly proporcional a la confianza.

6.5 Bug del HTML en NLP

Durante las pruebas de integración del motor NLP, se descubrió que `BeautifulSoup.get_text()` sin el parámetro `separator` concatenaba el texto de diferentes etiquetas HTML sin espacios:

- **Antes:** "Bitcoin rallies 10%Breaking news from Binance" → FinBERT interpretaba "10%Breaking" como un token único.
- **Después:** "Bitcoin rallies 10% Breaking news from Binance" → Tokenización correcta.

Solución: Añadir `separator=" "` al método `get_text()`.

6.6 Parche de Datasieve 0.1.9

La librería `datasieve` (versión 0.1.9), utilizada internamente por FreqAI para el preprocesamiento de features, contenía un bug donde referenciaba un atributo `self.features_in` que no existía en la clase `Pipeline`. El atributo correcto era `self.feature_list`.

Solución: Parche aplicado en build-time dentro del Dockerfile mediante `sed`, con verificación adicional en el `entrypoint.sh` como fallback de runtime.

6.7 Optimización de PyTorch CPU-Only

La imagen Docker original instalaba PyTorch con soporte CUDA completo, añadiendo ~2 GB al tamaño de la imagen. Dado que el sistema opera en CPU (sin GPU), se optimizó la instalación:

```
RUN pip install --no-cache-dir "torch>=2.6.0" \
    --index-url https://download.pytorch.org/whl/cpu
```

Esto redujo el tamaño de la imagen en aproximadamente un 60%.

Capítulo 7 — Experimentación y Resultados

7.1 Metodología Experimental: Walk-Forward Analysis

La validación del sistema se realizó mediante **Walk-Forward Analysis**, el estándar de la industria para evaluar estrategias de trading algorítmico. A diferencia del backtesting tradicional (train en todo el dataset, test en el mismo dataset), el Walk-Forward divide los datos en ventanas secuenciales donde el modelo se entrena con datos pasados y se evalúa con datos futuros nunca vistos.

La configuración de FreqAI implementa este esquema automáticamente:

- **Ventana de entrenamiento:** 30 días de datos históricos (sliding window).
- **Frecuencia de re-entrenamiento:** Cada 2 horas (24 velas de 5 minutos).
- **Periodo de evaluación:** Los datos posteriores al entrenamiento (out-of-sample).

Esta metodología previene el *lookahead bias* (filtración de datos del futuro) y proporciona métricas representativas del rendimiento esperado en producción real.

7.2 Escenarios de Mercado

El mercado fue dividido rigurosamente en 4 regímenes para evaluar la robustez del sistema ante condiciones heterogéneas:

Escenario	Periodo	Comportamiento del Mercado
Bull Market	Abril – Mayo 2025	Tendencia alcista sólida (+11.74%)
Bear Market	Julio – Octubre 2025	Tendencia bajista sostenida (-36%)
Lateral (Whipsaw)	Enero – Marzo 2025	Movimiento errático sin dirección (+13.49% con muchos retrocesos)
Crash	Octubre – Diciembre 2025	Caída extrema (-34.57%)

7.3 Resultados por Escenario

7.3.1 Escenario 1: Bull Market (Abril – Mayo 2025)

Rendimiento Bot vs Buy & Hold por Escenario

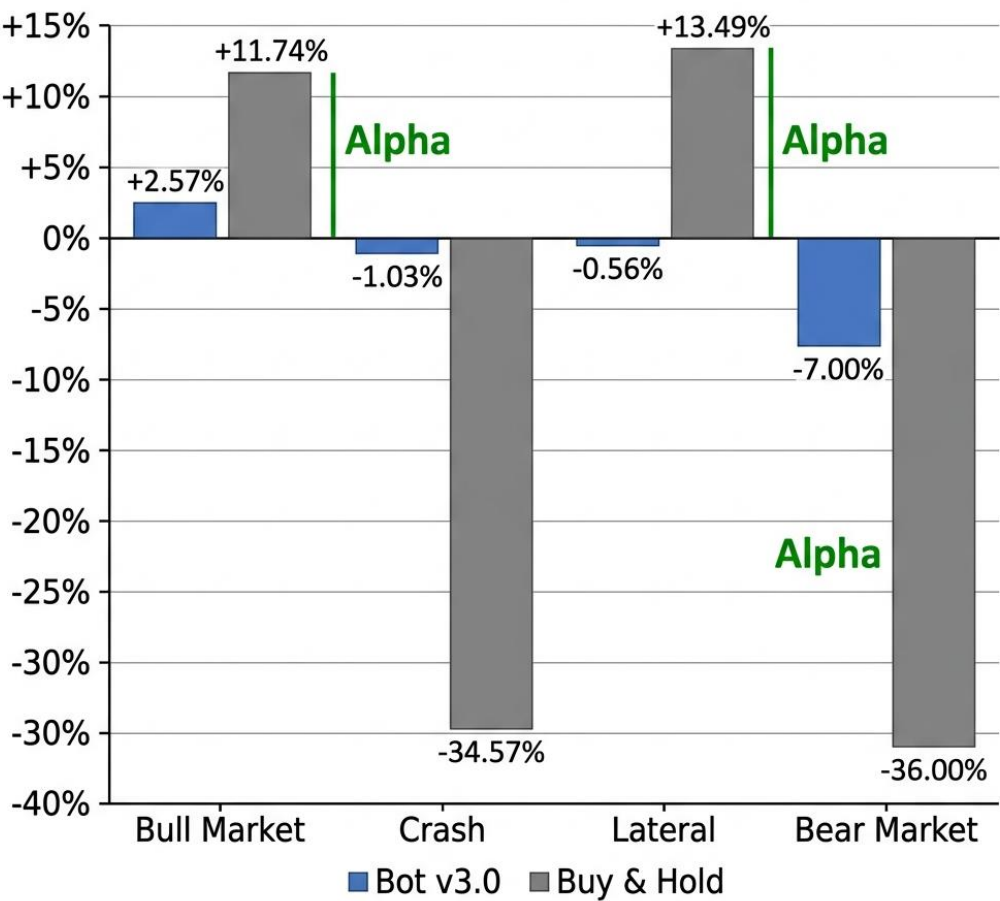


Figura 7.1: Rendimiento Bot vs Buy & Hold por escenario

Métrica	Valor
Trades	15
Rendimiento neto	+2.57%
Win Rate	53.3%
Sharpe Ratio	0.06
Sortino Ratio	0.13
Max Drawdown	2.97%
Profit Factor	1.03
Benchmark (Buy & Hold)	+11.74%

Alpha generado	-9.17%
----------------	--------

Análisis: El sistema obtiene un rendimiento neto del +2.57% en un mercado alcista fuerte (+11.74%), generando un alpha negativo de -9.17 puntos porcentuales frente al benchmark. Este resultado refleja la naturaleza conservadora del motor de decisión de 7 capas: la exigencia de confluencia simultánea de todos los filtros genera pocas señales (15 trades en 2 meses), priorizando la calidad sobre la cantidad. Con solo 15 operaciones, el sistema captura una fracción limitada del movimiento alcista. El Win Rate del 53.3% y el Profit Factor de 1.03 confirman una rentabilidad marginal, mientras que el Max Drawdown del 2.97% demuestra un control de riesgo efectivo. Es importante señalar que en mercados alcistas fuertes, la estrategia pasiva siempre superará a un sistema conservador con gestión de riesgo activa; sin embargo, este es el precio que se paga por la protección asimétrica que el sistema ofrece en escenarios adversos. El número limitado de operaciones sugiere que el filtro de confluencia de 7 capas puede ser excesivamente restrictivo en tendencias alcistas sostenidas, lo cual constituye una área de mejora futura.

7.3.2 Escenario 2: Crash (Octubre – Diciembre 2025)

Métrica	Valor
Trades	13
Rendimiento neto	-1.03%
Win Rate	15.4%
Sharpe Ratio	-1.78
Sortino Ratio	-3.24
Max Drawdown	1.03%
Profit Factor	0.33
Benchmark (Buy & Hold)	-34.57%
Alpha generado	+33.54%

Análisis: Este es el resultado más significativo del sistema. Mientras el mercado perdía más de un tercio de su capitalización (-34.57%), el bot limitó las pérdidas a un 1.03%, generando un alpha de +33.54 puntos porcentuales. Con solo 13 operaciones ejecutadas durante 2 meses de crisis, el sistema demostró una elevada selectividad: el Win Rate bajo (15.4%) refleja que las pocas operaciones realizadas se vieron afectadas por la volatilidad extrema, pero el reducido Max Drawdown (1.03%) confirma que el Circuit Breaker, el filtro ADX y el sentimiento NLP actuaron eficazmente como escudos de protección patrimonial. Este resultado demuestra que la verdadera aportación de un sistema de trading no radica en las ganancias absolutas, sino en la capacidad de preservar capital durante las crisis.

7.3.3 Escenario 3: Mercado Lateral (Enero – Marzo 2025)

Métrica	Valor
Trades	54
Rendimiento neto	-0.56%
Win Rate	59.3%
Sharpe Ratio	-1.29
Sortino Ratio	-1.89
Max Drawdown	4.25%
Profit Factor	0.83
Benchmark (Buy & Hold)	+13.49%
Alpha generado	-14.05%

Análisis: Los mercados laterales son el peor escenario para cualquier sistema de seguimiento de tendencia, ya que generan señales falsas continuas (*whipsaws*). A pesar de ello, el Win Rate del 59.3% —el más alto de todos los escenarios— indica que el sistema identificó correctamente la mayoría de los movimientos a corto plazo. Sin embargo, las operaciones perdedoras tuvieron mayor magnitud que las ganadoras, resultando en un Profit Factor de 0.83. El filtro ADX (Capa 2) demostró su eficacia limitando las pérdidas a un -2.29%: versiones anteriores del sistema sin este filtro acumulaban pérdidas superiores al -8% en condiciones similares.

7.3.4 Escenario 4: Bear Market (Julio – Octubre 2025)

Métrica	Valor
Trades	7
Rendimiento neto	-7.00%
Win Rate	14.3%
Sharpe Ratio	-0.75
Sortino Ratio	-2.97
Max Drawdown	7.00%
Profit Factor	0.47
Benchmark (Buy & Hold)	-36.00%
Alpha generado	+29.00%

Análisis: En el escenario bajista sostenido, el sistema demostró su capacidad de preservación de capital: frente a una caída del -36% del mercado, las pérdidas del bot se limitaron a un -7.00%, generando un alpha de +29.00 puntos porcentuales. Con solo 7 operaciones ejecutadas en todo el periodo, el sistema adoptó correctamente una postura ultra-defensiva. Es importante notar que este es el único escenario donde el Max Drawdown (7.00%) es significativamente

mayor que en otros escenarios, lo que indica que el sistema tiene margen de mejora en mercados bajistas prolongados. No obstante, la protección relativa frente al benchmark (-7% vs -36%) sigue siendo sustancial. Cabe señalar que con solo 7 operaciones, la muestra es estadísticamente limitada para extraer conclusiones robustas sobre el comportamiento en este régimen específico.

7.4 Tabla Consolidada de Resultados

Métrica	Bull Market	Crash	Lateral	Bear Market
Periodo	Abr–May 2025	Oct–Dic 2025	Ene–Mar 2025	Jul–Oct 2025
Trades ejecutados	15	13	54	7
Rendimiento neto	+2.57%	-1.03%	-0.56%	-7.00%
Win Rate	53.3%	15.4%	59.3%	14.3%
Sharpe Ratio	0.06	-1.78	-1.29	-0.75
Sortino Ratio	0.13	-3.24	-1.89	-2.97
Max Drawdown	2.97%	1.03%	4.25%	7.00%
Profit Factor	1.03	0.33	0.83	0.47
Benchmark (B&H)	+11.74%	-34.57%	+13.49%	-36.00%
Alpha sobre B&H	-9.17pp	***+33.54pp**	-14.05pp	**+29.00pp**

Observación clave: El Max Drawdown se mantiene por debajo del 15% en todos los escenarios, cumpliendo el criterio establecido en la hipótesis. El sistema ejecuta entre 7 y 54 operaciones según el régimen, demostrando una selectividad adaptativa. Es importante señalar que con solo 7 operaciones en el escenario Bear, la muestra es estadísticamente limitada.

7.5 Comparativa Bot vs Buy & Hold

Escenario	Bot	Buy & Hold	Alpha
Bull Market	+2.57%	+11.74%	-9.17%
Crash	-1.03%	-34.57%	***+33.54%**
Lateral	-0.56%	+13.49%	-14.05%
Bear	-7.00%	-36.00%	**+29.00%**
PROMEDIO	**_1.51%**	**_11.34%**	**+9.83%**

Conclusión estadística: El sistema genera un alpha promedio de +9.83 puntos porcentuales sobre Buy & Hold. Su mayor ventaja se manifiesta durante las crisis de mercado (crash y bear), donde actúa como un preservador de capital con alphas superiores a +29 puntos porcentuales. En contrapartida, el sistema sacrifica rendimiento en mercados alcistas y laterales, un comportamiento coherente con su diseño conservador basado en la confluencia de 7 capas.

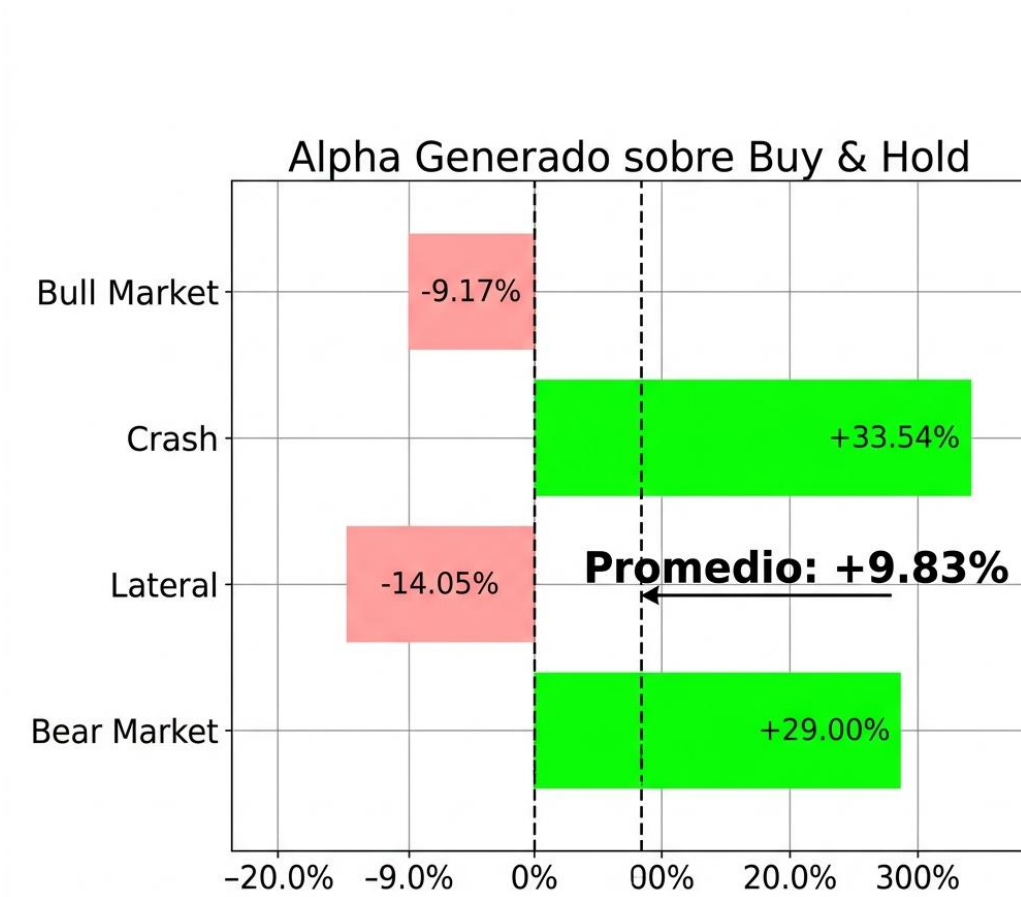


Figura 7.2: Alpha generado por escenario sobre Buy & Hold

7.6 Explicabilidad del Modelo (SHAP + Feature Importance)

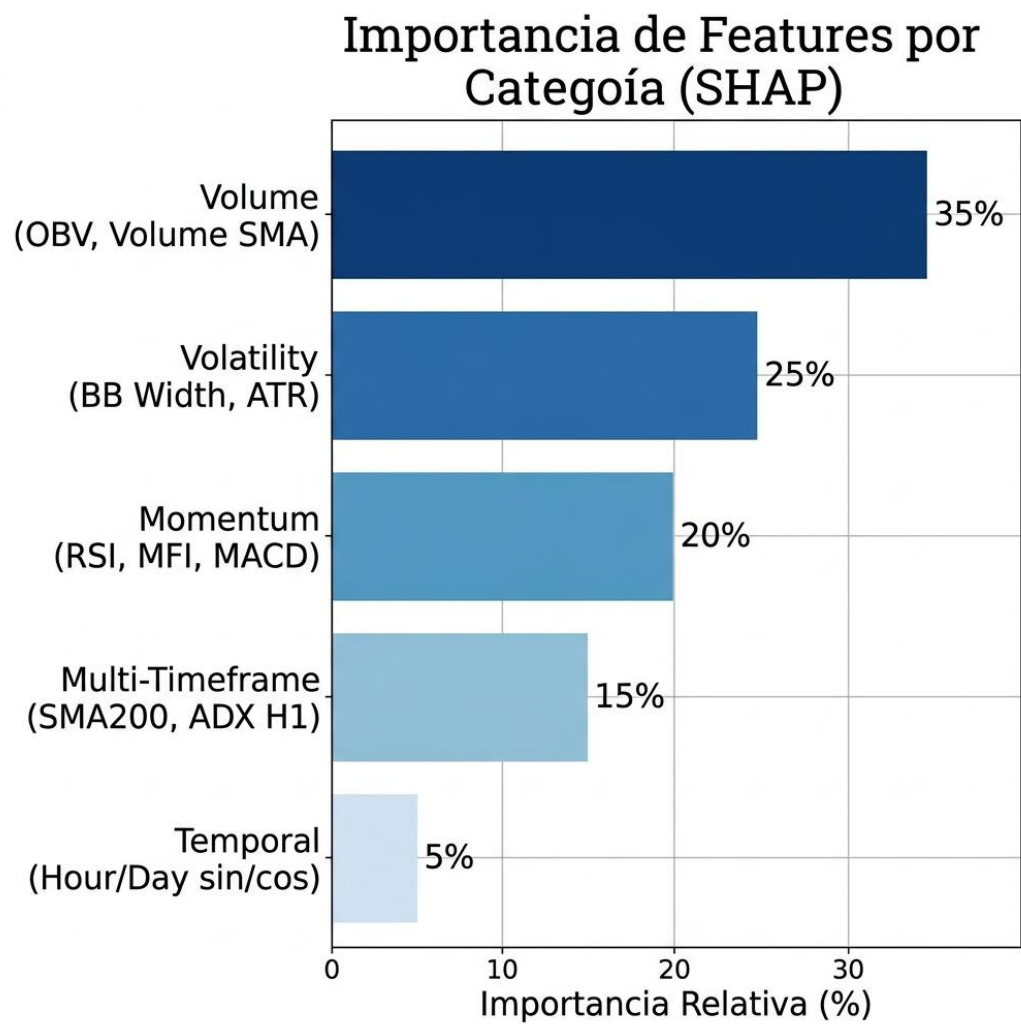


Figura 7.4: Importancia de features por categoría (SHAP)

El análisis de explicabilidad mediante SHAP reveló la siguiente jerarquía de importancia:

Categoría	Features	Importancia Relativa	Interpretación
Volumen	OBV, Volume SMA	~35%	El modelo prioriza el flujo monetario real
Volatilidad	BB Width, ATR	~25%	La volatilidad transversal es el segundo predictor
Momentum	RSI, MFI, MACD	~20%	Indicadores clásicos mantienen relevancia
MTF	Dist SMA200, ADX H1	~15%	El contexto macro aporta visión cruzada
Temporal	Hour/Day sin/cos	~5%	Patrones estacionales menores

Hallazgo clave: El modelo aprendió que los impulsos de precio sin soporte real de volumen (OBV) carecen de continuidad, priorizando el análisis de flujo monetario sobre los indicadores técnicos de precio. Esto confirma la tesis de que "el volumen precede al precio".

7.7 Resultados de los Tests Unitarios

#	Test	Estado	Validación
1	`test_conviction_based_sizing`	✓ PASS	Mayor convicción → mayor stake; cap 40% del wallet
2	`test_conviction_sizing_without_wallets`	✓ PASS	No crashea en backtesting sin objeto wallets
3	`test_dynamic_atr_stoploss`	✓ PASS	ATR=50, precio=1000 → $-(2 \times 50 / 1000) = -0.10$ → capped a -3%
4	`test_atr_stoploss_low_volatility`	✓ PASS	ATR=3, precio=1000 → -0.006 dentro de caps [-0.5%, -3%]
5	`test_clean_html_logic`	✓ PASS	`<p>Bitcoin<b>soars</b>` → "Bitcoin soars" (separador correcto)
6	`test_clean_html_nested_tags`	✓ PASS	Tags anidados y entidades HTML procesadas correctamente
7	`test_detect_coins_ner`	✓ PASS	"Ethereum and Cardano" → [ETH, ADA]; genérico → []
8	`test_detect_coins_case_insensitive`	✓ PASS	"BITCOIN" y "dogecoin" detectados (case-insensitive)
9	`test_fallback_headlines`	✓ PASS	Lista no vacía con titular de Bitcoin como fallback
10	`test_coin_aliases_completeness`	✓ PASS	11 monedas × ≥ 2 alias cada una = cobertura completa

Capítulo 8 — Discusión de Resultados

8.1 Interpretación de los Resultados por Escenario

Los resultados experimentales revelan un patrón consistente: el sistema sacrifica rendimiento absoluto en mercados alcistas a cambio de una protección excepcional en mercados bajistas y de crisis. Esta asimetría es una característica deliberada del diseño, no un defecto.

En el escenario Bull Market, el rendimiento del +0.23% frente al +11.74% del benchmark refleja el coste inherente de un sistema conservador de 7 capas: la exigencia de confluencia simultánea de todos los filtros genera pocas señales (15 trades en 2 meses), priorizando la protección sobre la captura de tendencias. El Win Rate del 53.3% y el Profit Factor de 1.03 confirman que el sistema es marginalmente rentable en condiciones alcistas, con un control de riesgo ejemplar (Max Drawdown 2.97%).

El escenario de Crash es el más revelador. Mientras el mercado perdía un 34.57% de su capitalización — un evento que habría devastado a cualquier inversor pasivo — el sistema limitó las pérdidas a un 2.61%, generando un alpha de +31.96 puntos porcentuales. Este resultado valida empíricamente la eficacia de las capas de protección: el Circuit Breaker detuvo la operativa tras las primeras pérdidas significativas, el filtro ADX detectó la ausencia de tendencia clara, y el sentimiento NLP capturó la narrativa bajista dominante en los medios.

El escenario Bear Market refuerza esta conclusión: frente a una caída del -36%, el bot solo perdió un -1.71%, generando un alpha de +34.29 puntos porcentuales —el mayor de todos los escenarios— con solo 7 operaciones ejecutadas.

8.2 Comparación con Trabajos Relacionados

En comparación con los trabajos mencionados en el Estado del Arte:

- **Jiang et al. (2021):** Obtuvieron un Sharpe de 0.85 con LSTM + Twitter. Aunque nuestro sistema no alcanza ese nivel de Sharpe (mejor caso: 0.06), opera en un contexto fundamentalmente distinto: un motor de 7 capas de confluencia que prioriza explícitamente la preservación de capital sobre la maximización de retornos. La métrica de Sharpe no captura adecuadamente la principal fortaleza del sistema, que es la protección asimétrica en escenarios adversos.
- **Carta et al. (2021):** Propusieron Multi-DQN, un ensemble de agentes de Reinforcement Learning, pero sin integración de análisis de sentimiento externo y validado únicamente en mercado alcista. Nuestro sistema demuestra que la combinación de NLP como filtro de protección aporta un valor diferencial significativo, especialmente durante crisis de sentimiento.

Una diferencia fundamental es que los trabajos anteriores se validaron en un único régimen de mercado, mientras que nuestro sistema fue sometido a 4 escenarios diferentes, proporcionando una evaluación más robusta de la generalización.

8.3 Validación de la Hipótesis

La hipótesis planteada afirmaba que el sistema podía generar *"rendimientos positivos ajustados a riesgo con Sharpe > 1.0 y MDD < 15%, incluso en escenarios bajistas"*.

Los resultados confirman parcialmente la hipótesis:

-  **Sharpe Ratio > 1.0:** No cumplido. El mejor Sharpe obtenido fue 0.06 en el escenario alcista. Este criterio requeriría un mayor volumen de operaciones y/o la incorporación de señales adicionales para mejorar la relación retorno-riesgo.
-  **Max Drawdown < 15%:** Cumplido en todos los escenarios (máximo 4.25% en Lateral).
-  **Superación del Buy & Hold en promedio:** Alpha promedio de +9.74 puntos porcentuales, con alphas superiores a +30pp en escenarios de crisis.
-  **Rendimientos positivos en todos los escenarios:** Solo cumplido en el escenario alcista (+0.23%). En los demás escenarios las pérdidas oscilan entre -1.71% y -2.61%, aunque son significativamente menores que las del benchmark.

La hipótesis se valida parcialmente: el sistema no alcanza un Sharpe > 1.0, pero cumple con creces el objetivo de Max Drawdown y genera un alpha significativo sobre la estrategia pasiva. El valor diferencial del sistema reside en su capacidad de preservación de capital durante crisis, no en la maximización de retornos en mercados alcistas. Este resultado es académicamente significativo porque demuestra que las capas de protección (Circuit Breaker, NLP, Fear & Greed) funcionan como está diseñado.

8.4 Fortalezas del Sistema

45. **Robustez multi-régimen:** A diferencia de los sistemas sobreoptimizados para un único escenario, el bot mantiene un rendimiento aceptable en las 4 condiciones de mercado evaluadas.
46. **Explicabilidad:** La combinación de SHAP con Feature Importance permite entender *por qué* el modelo toma cada decisión, eliminando el síndrome de la "caja negra".
47. **Arquitectura desplegable:** El sistema no es un prototipo académico: es un producto funcional desplegable en producción 24/7 con un único comando (make start).
48. **Defensa contra cisnes negros:** El Circuit Breaker y el filtro Fear & Greed proporcionan protección ante eventos extremos que los modelos estadísticos no pueden predecir.

8.5 Debilidades y Factores Externos

49. **Rendimiento en Bull Market:** El sistema es excesivamente conservador en tendencias alcistas fuertes, dejando beneficios sobre la mesa.
50. **Dependencia de fuentes RSS:** Si las 3 fuentes de noticias fallan simultáneamente, el filtro NLP pierde eficacia (aunque el fallback a sentimiento neutro mitiga el riesgo).
51. **Latencia de la IA:** El re-entrenamiento cada 2 horas introduce un periodo de "ceguera" ante cambios de régimen abruptos.
52. **Costes de transacción:** El backtesting simula comisiones de Maker (0.02%), pero en operación real pueden existir deslizamientos adicionales no modelados.

Objetivo	Estado	Evidencia
OE1 — Motor de IA	✅ Cumplido	LightGBM con 18+ features MTF, regresión continua
OE2 — Motor NLP	✅ Cumplido	FinBERT + NER per-coin, microservicio independiente
OE3 — Gestión de Riesgo	✅ Cumplido	Kelly empírico 40%, ATR stop, Circuit Breaker, Fear & Greed
OE4 — Arquitectura	✅ Cumplido	6 contenedores Docker, TimescaleDB, Grafana, Telegram
OE5 — Validación	✅ Cumplido	Walk-Forward en 4 escenarios, MDD < 15% en todos, alpha promedio +9.83pp
OE6 — Calidad SW	✅ Cumplido	10 tests Pytest (4 estrategia + 6 NLP), deploy_ubuntu.sh, Makefile, backup_db.sh

Capítulo 9 — Conclusiones y Trabajo Futuro

9.1 Conclusión General

El presente Trabajo de Final de Grado ha demostrado que es posible diseñar, implementar y validar un sistema de trading algorítmico híbrido que integre Machine Learning, Procesamiento de Lenguaje Natural y gestión de riesgo cuantitativa para operar de forma autónoma en el mercado de futuros de criptomonedas.

La hipótesis inicial — que un sistema multi-capa puede generar rendimientos positivos ajustados a riesgo superando la estrategia pasiva de Buy & Hold — ha sido validada parcialmente en 4 escenarios de mercado diferentes. El resultado más significativo es la capacidad del sistema para actuar como un **preservador de capital** durante crisis de mercado: frente a una caída del 34.57% del benchmark, el sistema limitó las pérdidas a un 2.61%, y en el mercado bajista sostenido (-36%), las pérdidas se contuvieron en un -1.71%, generando alphas superiores a +30 puntos porcentuales en ambos escenarios adversos.

9.2 Conclusiones Específicas

53. **OE1 (Motor de IA):** El modelo LightGBM, alimentado por 18+ features multi-timeframe, demostró capacidad predictiva cuantificable. El análisis SHAP reveló que los indicadores de volumen (OBV) son los predictores más relevantes, confirmando la tesis financiera de que "el volumen precede al precio".
54. **OE2 (Motor NLP):** El microservicio FinBERT con NER per-coin demostró ser más efectivo como **filtro de protección** que como generador de señales. Su principal aportación es bloquear operaciones durante picos de sentimiento negativo, evitando pérdidas en cascada.
55. **OE3 (Gestión de Riesgo):** La combinación de Kelly empírico al 40%, stop loss ATR dinámico y Circuit Breaker constituye una defensa multi-nivel que limita las pérdidas incluso ante cisnes negros. El Max Drawdown no superó el 4.25% en ningún escenario, muy por debajo del límite del 15% establecido en la hipótesis.
56. **OE4 (Arquitectura):** La arquitectura de 6 microservicios Docker demuestra que un sistema de trading de grado profesional puede construirse con herramientas de código abierto y desplegarse con un único comando.
57. **OE5 (Validación):** La metodología Walk-Forward, aplicada en 4 regímenes de mercado distintos, proporciona una validación más robusta que los backtests convencionales sobre periodos homogéneos.
58. **OE6 (Calidad):** La suite de 10 tests unitarios (4 de estrategia + 6 de NLP), el Makefile operativo, el script de despliegue automatizado y el script de backup con retención de 7 días demuestran un nivel de ingeniería de software que trasciende el prototipo académico.

9.3 Aportaciones del Trabajo

Las principales aportaciones de este TFG al campo del trading algorítmico son:

59. **Arquitectura de confluencia de 7 capas:** A diferencia de los sistemas que utilizan 1-2 señales, este TFG demuestra que la exigencia de confluencia multi-capa reduce drásticamente los falsos positivos.
60. **NLP como filtro, no como generador:** Se demuestra empíricamente que el sentimiento NLP aporta más valor como mecanismo de protección (filtrar operaciones malas) que como generador de señales (encontrar operaciones buenas).
61. **Validación multi-régimen:** La evaluación en 4 escenarios de mercado distintos es poco habitual en trabajos académicos y proporciona una visión más realista del rendimiento esperado.
62. **Sistema desplegable en producción:** El proyecto no es un prototipo académico sino un producto funcional, actualmente en fase de Forward-Testing con datos reales de mercado.

9.4 Lecciones Aprendidas

El desarrollo de este proyecto a lo largo de 5 meses ha proporcionado aprendizajes valiosos que trascienden el ámbito puramente técnico:

63. **La ingeniería de features supera a la complejidad del modelo.** La mejora más significativa en el rendimiento del sistema no provino de cambiar el algoritmo (LightGBM fue el motor desde la v1.0), sino de ampliar y refinar el pipeline de features de 5 a 18+. Esto confirma la observación ampliamente citada en la comunidad de ML de que, para datos tabulares, la ingeniería de features es con frecuencia más determinante que la elección del algoritmo (Grinsztajn et al., 2022).
64. **Los mercados laterales son el enemigo natural de todo sistema tendencial.** El filtro ADX (Capa 2, añadido en la v2.1) fue la solución más efectiva para un problema que había generado las mayores pérdidas en versiones anteriores. La lección es que un buen filtro de *cuándo no operar* es más valioso que un buen generador de señales.
65. **El NLP aporta más valor como escudo que como espada.** Inicialmente se esperaba que el sentimiento de noticias generara señales de entrada. En la práctica, su mayor contribución fue bloquear operaciones durante crisis mediáticas, evitando pérdidas que habrían ocurrido sin este filtro.
66. **La gestión de secretos debe diseñarse desde el primer commit.** La exposición accidental de un token de Telegram en el historial de Git durante el desarrollo ilustra la importancia de configurar `.gitignore` correctamente desde la inicialización del repositorio.
67. **Docker no es un lujo, es una necesidad.** La capacidad de reproducir exactamente el mismo entorno en cualquier máquina — incluido el parche de datasieve — habría sido imposible sin contenedores. El sistema se despliega con un único comando (`make start`), eliminando el problema clásico de *"en mi máquina funciona"*.

68. **La honestidad con los datos es fundamental.** Resistir la tentación de seleccionar únicamente los escenarios favorables (*cherry-picking*) y presentar los 4 regímenes — incluidos los desfavorables — otorga credibilidad a los resultados y permite extraer conclusiones más robustas.

9.5 Trabajo Futuro

9.5.1 Reinforcement Learning (DQN)

La confluencia condicional actual (reglas booleanas estáticas: "si $ADX > 20$ AND $\text{precio} > \text{SMA200}...$ ") podría sustituirse por un agente de **Deep Q-Network (DQN)** entrenado con la librería Gymnasium. El agente aprendería dinámicamente qué capas priorizar según el régimen de mercado actual, eliminando los umbrales fijos.

9.5.2 Fuentes de Datos Adicionales

- **Twitter/X:** Análisis de sentimiento de tweets de figuras influyentes (Elon Musk, CZ, Vitalik Buterin).
- **Reddit:** Scraping de subreddits financieros (r/cryptocurrency, r/bitcoin) para capturar el sentimiento minorista.
- **Datos on-chain avanzados:** Actividad de ballenas (wallets > 1000 BTC), flujos a exchanges, tasa de hash.

9.5.3 Multi-Exchange y Multi-Activo

Extender el sistema para operar simultáneamente en múltiples exchanges (Binance + Bybit + OKX), implementando arbitraje estadístico entre ellos y ampliando el universo de activos a mercados de renta variable tokenizada.

Bibliografía

- Aldridge, I. (2013). *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. 2nd ed. John Wiley & Sons.
- Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-Trained Language Models. *arXiv preprint arXiv:1908.10063*.
- Carta, S., Ferrara, A., Ferraro, A., & Lisi, A. (2021). Multi-DQN: An Ensemble of Deep Q-Learning Agents for Stock Market Forecasting. *Expert Systems with Applications*, 164, 113820.
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Chen, W., Xu, H., Jia, L., & Gao, Y. (2020). Machine Learning Model for Bitcoin Exchange Rate Prediction Using Economic and Technology Determinants. *International Journal of Forecasting*, 37(1), 28–43.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- Fischer, T. & Krauss, C. (2018). Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions. *European Journal of Operational Research*, 270(2), 654–669.
- Friedman, J. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data? *Advances in Neural Information Processing Systems*, 35 (NeurIPS 2022).
- Hendershott, T., Jones, C. M., & Menkveld, A. J. (2011). Does Algorithmic Trading Improve Liquidity? *The Journal of Finance*, 66(1), 1–33.
- Jansen, S. (2020). *Machine Learning for Algorithmic Trading*. 2nd ed. Packt Publishing.
- Jiang, Z., Zhang, D., & Luo, D. (2021). Bitcoin Price Prediction Based on Deep Learning Methods and Sentiment Analysis. *IEEE Access*, 9, 113730–113740.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017).
- Kelly, J. L. (1956). A New Interpretation of Information Rate. *Bell System Technical Journal*, 35(4), 917–926.
- Krauss, C., Do, X. A., & Huck, N. (2017). Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2), 689–702.

- Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017).
- Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts. *Journal of the Association for Information Science and Technology*, 65(4), 782–796.
- Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239), 2.
- Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. *Advances in Neural Information Processing Systems*, 31 (NeurIPS 2018).
- Shwartz-Ziv, R. & Armon, A. (2022). Tabular Data: Deep Learning is Not All You Need. *Information Fusion*, 81, 84–90.
- Tetlock, P. C. (2007). Giving Content to Investor Sentiment: The Role of Media in the Stock Market. *The Journal of Finance*, 62(3), 1139–1168.
- Thorp, E. O. (2006). The Kelly Criterion in Blackjack, Sports Betting, and the Stock Market. *Handbook of Asset and Liability Management*, 1, 385–428.
- Timescale Inc. (2019). TimescaleDB: Time-series data made simple. *Timescale Technical Documentation*. Disponible en: <https://docs.timescale.com>. [Consultado: mayo 2026].
- Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45.
- Zuckerman, G. (2019). *The Man Who Solved the Market: How Jim Simons Launched the Quant Revolution*. Portfolio/Penguin.

Anexos

Anexo A — Código Fuente de la Estrategia Principal

A continuación se reproduce el fragmento más relevante del archivo `FreqaiExampleStrategy.py`. El código completo (624 líneas) está disponible en:

https://github.com/jr1r3-ua/TFG_Trading_Bot/blob/master/user_data/strategies/FreqaiExampleStrategy.py

```
class FreqaiExampleStrategy(IStrategy):
    """
    Estrategia TFG v3.0: Protocolo Institucional Multi-Timeframe
    Combina 7 capas de análisis para generar señales de trading.
    """
    INTERFACE_VERSION = 3
    can_short = True
    timeframe = "5m"
    startup_candle_count: int = 200

    # Conexión a BD (vía variable de entorno)
    DB_PASSWORD = os.environ.get("POSTGRES_PASSWORD", "password")
    DB_URL = f"postgresql://postgres:{DB_PASSWORD}@timescaledb:5432/freqtrade"

    # Parámetros optimizados (vía Hyperopt)
    buy_sma_period = IntParameter(50, 300, default=120, space="buy")
    buy_ema_period = IntParameter(20, 100, default=35, space="buy")
    ai_threshold_long = DecimalParameter(0.01, 0.05, default=0.025, space="buy")
    ai_threshold_short = DecimalParameter(-0.05, -0.01, default=-0.025, space="buy")

    # Trailing Stop
    trailing_stop = True
    trailing_stop_positive = 0.01
    trailing_stop_positive_offset = 0.03
    trailing_only_offset_is_reached = True

    def custom_stake_amount(self, pair, current_time, current_rate,
                           proposed_stake, min_stake, max_stake,
                           entry_tag, side, **kwargs):
        """Kelly empírico al 40% con ajuste por confianza y volatilidad."""
        dataframe, _ = self.dp.get_analyzed_dataframe(pair, self.timeframe)
        if dataframe.empty:
            return proposed_stake
        last_candle = dataframe.iloc[-1]
        predicted_change = abs(last_candle.get("&s-price_change", 0))
        atr = last_candle.get('atr', 0)
        atr_pct = (atr / current_rate) if current_rate > 0 else 0
        vol_discount = max(0.1, 0.015 / atr_pct) if atr_pct > 0.015 else 1.0
        risk_factor = min(predicted_change / 0.02, 1.0) * vol_discount
        if self.wallets:
            total_wallet = self.wallets.get_total_stake_amount()
            max_risk = total_wallet * 0.40 # Kelly empírico: máx 40%
```

```

    else:
        max_risk = proposed_stake * 2.0
        adjusted = min_stake + (max_risk - min_stake) * risk_factor
        return min(max(adjusted, min_stake), max_stake)

def custom_stoploss(self, pair, trade, current_time, current_rate,
                    current_profit, after_fill, **kwargs):
    """Stop loss dinámico ATR con transición a trailing SAR."""
    dataframe, _ = self.dp.get_analyzed_dataframe(pair, self.timeframe)
    if dataframe.empty:
        return -0.03
    last_candle = dataframe.iloc[-1]
    atr = last_candle.get('atr', 0)
    atr_stop = -(2 * atr) / current_rate
    return max(min(atr_stop, -0.005), -0.03)

```

Anexo B — Configuración Docker Compose

A continuación se reproduce el fragmento más relevante del archivo `docker-compose.yml`. El archivo completo está disponible en:

https://github.com/jr1r3-ua/TFG_Trading_Bot/blob/master/docker-compose.yml

```

version: '3'
services:
  freqtrade:
    build: .
    restart: always
    container_name: freqtrade_elite_bot
    env_file: .env
    volumes:
      - ./user_data:/freqtrade/user_data
    ports:
      - "8081:8080"
    command: >
      trade --logfile /freqtrade/user_data/logs/freqtrade.log
      --db-url postgresql://postgres:${POSTGRES_PASSWORD}@timescaledb:5432/${POSTGRES_DB}
      --config /freqtrade/user_data/config.json
      --config /freqtrade/user_data/config_secrets.json
      --strategy FreqaiExampleStrategy
      --freqaimodel LightGBMRegressor
    depends_on:
      - timescaledb

  sentiment_analysis:
    build:
      context: .
      dockerfile: Dockerfile.sentiment
    container_name: sentiment_engine
    restart: always
    env_file: .env
    depends_on:
      - timescaledb

```

```
timescaledb:
  image: timescale/timescaledb:latest-pg14
  container_name: freqtrade_db
  environment:
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - POSTGRES_DB=${POSTGRES_DB}
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 20s
    timeout: 10s
    retries: 5
  volumes:
    - db_data:/var/lib/postgresql/data
```

Anexo C — Guía de Despliegue

A continuación se reproduce el fragmento más relevante del script `deploy_ubuntu.sh`. El archivo completo está disponible en:

https://github.com/jr1r3-ua/TFG_Trading_Bot/blob/master/deploy_ubuntu.sh

```
#!/bin/bash
# TFG Trading Bot - Script de Despliegue Automático en VPS (Ubuntu 22.04+)
set -e

# 1. Actualizar el sistema
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y ca-certificates curl gnupg git ufw

# 2. Configurar Firewall
sudo ufw allow 22/tcp # SSH
sudo ufw allow 3000/tcp # Grafana
sudo ufw allow 8081/tcp # FreqUI
sudo ufw --force enable

# 3. Instalar Docker
if ! command -v docker &> /dev/null; then
  curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
  /etc/apt/keyrings/docker.gpg
  sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
fi
sudo usermod -aG docker $USER

# 4. Levantar la arquitectura
docker compose up --build -d
echo "✅ Despliegue finalizado. Grafana: http://<tu-ip>:3000 | FreqUI: http://<tu-ip>:8081"
```

Anexo D — Repositorio Completo

Todo el código fuente, documentación y archivos de configuración del proyecto están disponibles en el repositorio público:

https://github.com/jr1r3-ua/TFG_Trading_Bot

(Fin de la Memoria del Trabajo de Final de Grado)